МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ _Тебенькова Кристина Сергеевна_____
(подпись) (фамилия, имя, отчество)

Группы ИПс-31

Руководитель практической подготовки от профильной организации

_____директор_____
(должность)

_____ _____Грищенко Владимир Николаевич_____
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ _____
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

Введение 2
Раздел 1. Организационно-подготовительный этап прохождения практики на предприятии 5
1.1 Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии 5
1.2. Описание рабочего места, используемой техники и программного обеспечения 7
Раздел 2. Ознакомление со структурой и характером деятельности подразделения 9
2.1 Ознакомление с организацией работы на предприятии или в структурном подразделении 9
2.2. Ознакомление с должностными и функциональными обязанностями 10
2.3. Организация информационного обеспечения и защиты информации предприятия 11
2.4. Структурные и функциональные схемы предприятия 12
РАЗДЕЛ 3. Работа на рабочих местах или в подразделении предприятия 14
3.1. Выполнение производственных заданий 14
3.1.1. Использование выбранной системы контроля версий 14
3.1.2. Разработка программного обеспечения 16
3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества 29
3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования 31
3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств 32
3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения 34
3.1.7. Интегрирование программных модулей 37
3.1.8. Верификация и аттестация программного обеспечения 39
Заключение 43
Список используемых источников 45
Приложения 49

Введение

Производственная практика является важнейшим этапом подготовки специалистов, направленным на комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

Основными видами деятельности в ходе практики являлись: разработка требований к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; моделирование процесса разработки программного обеспечения; проектирование архитектуры приложения; реализация программных модулей; проведение отладки и тестирования; интеграция компонентов в единую систему.

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;

2. Выполнять отладку программного модуля с использованием специализированных программных средств;

3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;

4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

5. Выбирать способы решения задач профессиональной деятельности применительно к различным контекстам;

6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности;

7. Планировать и реализовывать собственное профессиональное и личностное развитие;

8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами;

9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста;

10. Использовать информационные технологии в профессиональной деятельности;

11. Пользоваться профессиональной документацией на государственном и иностранном языках.

Для решения поставленных задач была разработана информационная система для автоматизации учета партнеров, товаров и складских запасов ООО «Цветы Прикамья». В процессе создания системы применялись современные технологии разработки: язык программирования C#, платформа .NET, технология WPF для создания пользовательского интерфейса, СУБД PostgreSQL и Entity Framework Core для работы с данными.

В настоящем отчете представлены результаты прохождения производственной практики, описание выполненных работ, применяемых методов и технологий, а также анализ полученных результатов.

Раздел 1. Организационно-подготовительный этап прохождения практики на предприятии

1.1 Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

В рамках первого этапа производственной практики в ООО «Цветы Прикамья» был проведен вводный инструктаж, целью которого являлось ознакомление с организационной структурой предприятия, основными принципами трудового распорядка и комплексом мер, направленных на обеспечение безопасности в процессе профессиональной деятельности.

1. Общие функциональные обязанности практиканта:
   - Выполнение задач и поручений, делегированных непосредственным руководителем от предприятия (привлеченным специалистом по ИТ), в соответствии с программой практики и индивидуальным планом.
   - Соблюдение установленного на предприятии режима работы, трудовой и производственной дисциплины.

     Изучение и неукоснительное следование правилам внутреннего распорядка, положениям по охране труда и технике безопасности.
   - Освоение технологических процессов, связанных с функционированием ИТ-инфраструктуры, и используемого программно-технического обеспечения.
   - Своевременное и качественное выполнение запланированных работ, оформление необходимой отчетной документации.

2. Общие правила техники безопасности на предприятии:
   - Запрещено находиться на территории предприятия в состоянии алкогольного, наркотического или токсического опьянения.
   - Курить разрешается только в специально отведенных и оборудованных для этого местах (согласно политике предприятия, курение допускается только за пределами здания).

- Знать и уметь пользоваться первичными средствами пожаротушения (огнетушители), знать расположение аварийных выходов и путей эвакуации.
- Запрещается самостоятельно предпринимать действия для ликвидации аварийных ситуаций. О любой нештатной ситуации (запах гари, задымление, искрение электропроводки) необходимо немедленно сообщить непосредственному руководителю.
- Соблюдать чистоту и порядок на рабочих местах, не загромождать проходы и эвакуационные пути.

3. Правила техники безопасности на конкретном рабочем месте:

Перед началом работы на рабочем месте, оснащенном персональным компьютером, были изучены следующие правила:

- Перед началом работы необходимо визуально убедиться в исправности рабочего оборудования (системный блок, монитор, клавиатура, мышь), а также в целостности электропроводки и розеток.
- Рабочее место должно быть организовано эргономично: монитор расположен на расстоянии вытянутой руки, его верхний край находится на уровне или чуть ниже уровня глаз, обеспечивается возможность опоры для спины и предплечий.
- Запрещается размещать на рабочих столах и вблизи электрооборудования посторонние предметы, в особенности жидкости (чашки с чаем, бутылки с водой), во избежание пролива и короткого замыкания.
- Экран монитора не должен быть обращен в сторону окна или ярких источников света для предотвращения бликов. Необходимо обеспечить равномерное и достаточное освещение рабочей зоны.

4. Правила безопасности при работе с электрическими приборами и устройствами:

- Запрещается самостоятельно разбирать, ремонтировать или вскрывать корпуса любой оргтехники (компьютеры, принтеры, ИБП, сетевые

фильтры), находящейся под напряжением. Все работы по ремонту и обслуживанию техники производятся только при отключенном питании.

- Подключение и отключение периферийных устройств (принтер, сканер, внешний накопитель) должно производиться при выключенном питании оборудования.
- Не допускается подключение неисправного оборудования, а также использование электроприборов с видимыми повреждениями корпуса, кабеля питания или вилки.
- Запрещается прикасаться к задним панелям системных блоков, клеммам и токоведущим частям оборудования, находящегося под напряжением.
- При обнаружении таких признаков неисправности, как искрение, запах гари, несвойственный шум или повышенный нагрев оборудования, необходимо немедленно отключить устройство от электросети и сообщить об этом руководителю.

Итог инструктажа:

В результате проведения вводного и первичного инструктажа были изучены и приняты к исполнению ключевые нормы и правила, обеспечивающие безопасные условия труда и эффективную организацию рабочего процесса в период прохождения производственной практики в ООО «Цветы Прикамья». Факт прохождения инструктажа зафиксирован в журнале регистрации инструктажей.

1.2. Описание рабочего места, используемой техники и программного обеспечения

В рамках организационно-подготовительного этапа мною было проведено описание рабочего места, предоставленного для прохождения практики, а также анализ используемой на предприятии вычислительной техники и программного обеспечения.

Характеристика рабочего места:

Рабочее место практиканта было организовано в административном помещении ООО «Цветы Прикамья» и оснащено персональным компьютером, подключенным к локальной сети предприятия и имеющим доступ в сеть Интернет.

Технические характеристики компьютера:

- Процессор: Intel Core i5-10400 (6 ядер, 12 потоков, 2.9 ГГц)
- Оперативная память: 8 ГБ DDR4
- Накопитель: SSD 256 ГБ (для операционной системы и ПО), HDD 500 ГБ (для хранения данных)
- Операционная система: Windows 10 Pro (лицензионная версия)
- Монитор: 24" LG, разрешение 1920x1080
- Периферийные устройства: клавиатура, мышь (проводные), принтер HP LaserJet P1102 (сетевой доступ).

Программное обеспечение, установленное на рабочем месте и используемое в деятельности предприятия:

1. Системное ПО:
   - Операционная система Windows 10 Pro;
   - Архиватор 7-Zip;
   - Браузеры: Google Chrome, Яндекс.Браузер.
2. Прикладное ПО общего назначения:
   - Microsoft Office 2019 (Word, Excel, Outlook) — для работы с документами, таблицами и электронной почтой;
   - Adobe Acrobat Reader — для просмотра PDF-документации.
3. Специализированное ПО:
   - 1С:Бухгалтерия 8.3 — для ведения бухгалтерского и налогового учёта;
   - Клиент-Банк (система удаленного доступа к расчетному счету организации).
4. Средства защиты информации:
   - Kaspersky Endpoint Security — антивирусная защита рабочих станций.

Раздел 2. Ознакомление со структурой и характером деятельности подразделения

2.1 Ознакомление с организацией работы на предприятии или в структурном подразделении

В рамках изучения деятельности ООО «Цветы Прикамья» мною была проанализирована структура предприятия, характер его деятельности, а также организация трудовых процессов.

1. Краткая характеристика предприятия:

Общество с ограниченной ответственностью «Цветы Прикамья» зарегистрировано 19 ноября 2010 года в г. Перми. Основным видом деятельности компании является деятельность по благоустройству ландшафта (код ОКВЭД 81.30), а также розничная торговля цветами, семенами, посадочным материалом, удобрениями и средствами защиты растений.

Предприятие имеет действующий салон цветов, расположенный по адресу: г. Пермь, ул. Кронштадтская, 39в (Дзержинский район). Салон работает ежедневно: в будние дни с 09:00 до 20:00, в субботу с 10:00 до 19:00, в воскресенье с 10:00 до 18:00.

2. Структура предприятия и ИТ-подразделения:

ООО «Цветы Прикамья» относится к категории микропредприятий со среднесписочной численностью сотрудников 11 человек. Управление компанией осуществляет директор — Грищенко Владимир Николаевич.

В связи с небольшим размером организации, отдельное штатное ИТ-подразделение не предусмотрено. Функции по поддержке и развитию информационной инфраструктуры возложены на:

- привлеченного специалиста (внешний аутсорсинг)
- либо на одного из сотрудников, совмещающего обязанности системного администратора

В период прохождения практики взаимодействие осуществлялось с привлеченным ИТ-специалистом, который координировал выполнение работ по настройке и сопровождению программного обеспечения.

3. Описание функций подразделений и их взаимодействие:

Основные подразделения предприятия:

- Административно-управленческий персонал (директор, бухгалтер) — отвечает за общее руководство, финансовый учёт и документооборот.
- Производственный отдел (озеленители, рабочие) — выполняет основные виды деятельности по благоустройству и озеленению.
- ИТ-сопровождение (внешний специалист) — обеспечивает бесперебойную работу компьютерной техники, серверов, сетевого оборудования и программного обеспечения.

Взаимодействие между подразделениями осуществляется через локальную сеть и общие папки (файловый сервер), а также с помощью электронной почты и мессенджеров.

4. Режим работы и формы организации труда:

- Режим работы: пятидневная рабочая неделя с 9:00 до 18:00.
- Форма организации труда: повременная (для офисных сотрудников) и сдельная (для производственного персонала).

2.2. Ознакомление с должностными и функциональными обязанностями

В ходе практики мною были изучены должностные обязанности специалиста, отвечающего за ИТ-инфраструктуру предприятия (привлеченного ИТ-специалиста), а также составлен перечень его прав и обязанностей.

1. Должностная инструкция сотрудника ИТ-подразделения:

Хотя отдельного штатного ИТ-подразделения в компании нет, функциональные обязанности по обслуживанию вычислительной техники и ПО закреплены в договоре аутсорсинга либо в должностной инструкции сотрудника, совмещающего эти функции. Основные обязанности включают:

- обеспечение бесперебойной работы компьютерного оборудования и оргтехники;
- установка, настройка и обновление операционных систем и прикладного ПО;
- администрирование баз данных (в том числе 1С:Бухгалтерия);
- обеспечение антивирусной защиты и информационной безопасности;
- создание резервных копий данных и их восстановление при необходимости;
- консультирование сотрудников по вопросам работы с ПО и оргтехникой.

2. Права и обязанности специалиста:

Специалист имеет право:

- запрашивать у сотрудников информацию, необходимую для выполнения служебных задач;
- вносить предложения по модернизации программно-технического обеспечения;
- ограничивать доступ пользователей к ресурсам при нарушении политики безопасности.

Обязан:

- соблюдать трудовую дисциплину;
- обеспечивать сохранность служебной информации;
- своевременно реагировать на сбои и нештатные ситуации.

2.3. Организация информационного обеспечения и защиты информации предприятия

1. Используемые информационные системы и программные средства:

На предприятии применяются следующие информационные системы:

- 1С:Бухгалтерия 8.3 — для ведения бухгалтерского и налогового учёта;
- Клиент-Банк (система удалённого доступа к расчётному счёту) — для работы с финансовыми операциями;

- Файловое хранилище на базе локальной сети — для обмена документами между сотрудниками;
- Microsoft Office 2019 (Outlook) — для работы с электронной почтой и документацией.

2. Процессы сбора, регистрации и обработки информации:
- Сбор первичной информации осуществляется сотрудниками вручную (накладные, акты выполненных работ, договоры).
- Регистрация данных производится в системе 1С и в электронных таблицах Excel.
- Обработка информации включает формирование отчётности, анализ данных и подготовку документов для контролирующих органов.

3. Методология проектирования и эксплуатации ИС:

В связи с использованием типовой конфигурации «1С:Бухгалтерия», методология эксплуатации соответствует рекомендациям фирмы «1С». Проектирование и доработка системы осуществляются силами привлечённых специалистов при необходимости [7].

4. Применяемые методы защиты информации:
- разграничение прав доступа к папкам и файлам в локальной сети;
- использование парольной политики на вход в систему;
- наличие лицензионного антивируса Kaspersky Endpoint Security;
- регулярное резервное копирование баз данных 1С на внешний носитель.

2.4. Структурные и функциональные схемы предприятия

1. Требования к техническим и программным средствам:

Для обеспечения бесперебойной работы предприятия к техническим и программным средствам предъявляются следующие требования:
- лицензионная чистота используемого ПО;
- регулярное обновление операционных систем и прикладных программ;
- достаточная производительность оборудования для работы с 1С и офисными приложениями;

- наличие резервного копирования критически важных данных.

2. Анализ используемых средств защиты информации:

В ходе анализа было установлено:

- на всех рабочих станциях установлен антивирус Kaspersky Endpoint Security, базы регулярно обновляются;
- используется встроенный межсетевой экран Windows;
- резервное копирование баз 1С производится ежедневно на внешний жёсткий диск;
- выявлена потенциальная уязвимость — использование простых паролей отдельными сотрудниками (рекомендовано усиление парольной политики).

3. Ознакомление с делопроизводством и ИТ-документацией:

Мною были изучены:

- журнал учёта выдачи ключей и доступа в помещения;
- документация на приобретённое оборудование;
- инструкции пользователей по работе с 1С и офисным ПО.

4. Применяемые технологии безопасности:

- парольная защита BIOS и учётных записей пользователей;
- разграничение доступа к общим папкам;
- шифрование данных на съёмных носителях не применяется, однако резервные копии хранятся в сейфе у бухгалтера.

РАЗДЕЛ 3. Работа на рабочих местах или в подразделении предприятия

3.1. Выполнение производственных заданий

3.1.1. Использование выбранной системы контроля версий

В процессе разработки программного обеспечения мною использовалась система контроля версий Git. Выбор данной системы обусловлен ее популярностью, бесплатностью, возможностью распределенной работы, надежностью и интеграцией со средой разработки Visual Studio.

Git – это распределенная система управления версиями, которая позволяет отслеживать изменения в файлах, хранить историю изменений, возвращаться к предыдущим версиям, работать над проектом нескольким разработчикам одновременно. Основные преимущества Git:

- Локальный репозиторий – каждый разработчик имеет полную копию проекта на своем компьютере;
- Ветвление (branching) – возможность создавать отдельные ветки для разработки новых функций;
- Слияние (merging) – объединение изменений из разных веток;
- Фиксация изменений (commit) – сохранение состояния проекта с комментарием;
- Отслеживание истории – возможность просмотра всех изменений и возврата к любой версии.

Процесс работы с Git в ходе практики:

1. Инициализация репозитория:
   - Создана локальная папка для проекта;
   - Выполнена команда `git init` для инициализации репозитория.
2. Создание структуры проекта:
   - Созданы папки для разных слоев приложения (Models, Data, Services, WPF);
   - Добавлены начальные файлы.
3. Добавление файлов в отслеживание:

- Команда git add . – добавление всех измененных файлов в индекс;
- Команда git add <filename> – добавление конкретного файла.

4. Фиксация изменений (коммиты):
   - После каждого значимого этапа выполнялась фиксация изменений командой git commit -m "описание";
   - Комментарии к коммитам составлялись на русском или английском языке с четким описанием выполненных изменений.
5. Создание веток:
   - Для разработки отдельных модулей создавались ветки командой git branch <имя_ветки>;
   - Переключение между ветками командой git checkout <имя_ветки>.
6. Созданные ветки:
   - feature/partners – разработка модуля партнеров;
   - feature/products – разработка модуля товаров;
   - feature/warehouse – разработка складского модуля;
   - feature/reports – разработка отчетов;
   - feature/tests – написание тестов.
7. Слияние веток:

   После завершения разработки модуля ветка сливалась с основной веткой main командой git merge <имя_ветки>.
8. Просмотр истории:
   - Команда git log – просмотр истории коммитов;
   - Команда git status – просмотр текущего состояния файлов.

Использование Git позволило:

- Сохранять все версии разрабатываемого приложения;
- При необходимости откатываться к предыдущим рабочим версиям;
- Экспериментировать с новыми функциями в отдельных ветках без риска повредить основной код;
- Вести структурированную историю разработки;
- Обеспечить резервное копирование кода.

3.1.2. Разработка программного обеспечения

Выбор модели разработки

Для разработки информационной системы была выбрана итерационная модель. Данный выбор обоснован следующими факторами:

Сравнительный анализ моделей разработки:

Модель	Преимущества	Недостатки
Каскадная	Простота, четкая структура этапов	Невозможность вернуться к предыдущему этапу
V-образная	Тестирование на ранних этапах	Жесткая структура, сложность изменений
Прототипирования	Быстрое получение обратной связи	Риск создания "черновика"
RAD (быстрая разработка)	Высокая скорость разработки	Требует высокой квалификации команды
Итерационная	Гибкость, возможность изменений, раннее тестирование	Требует четкого планирования итераций
Спиральная	Управление рисками	Сложность, высокая стоимость

Обоснование выбора итерационной модели:

1. Разделение на модули: Проект четко разделен на логические модули (управление партнерами, управление товарами, складской учет, отчетность), что позволяет разрабатывать их последовательно.
2. Гибкость: В процессе разработки могли уточняться требования, вноситься изменения – итерационная модель позволяет это делать без срыва сроков.
3. Раннее тестирование: Каждая итерация завершалась тестированием, что позволяло выявлять ошибки на ранних этапах.
4. Демонстрация результатов: После каждой итерации можно было показать промежуточный результат руководителю практики и получить обратную связь.
5. Снижение рисков: Поэтапная реализация снижает риск невыполнения проекта в целом.

Инструменты, использованные для создания продукта

Инструмент	Назначение	Характеристика
Visual Studio 2022	Интегрированная среда разработки	Бесплатная, мощная среда с поддержкой C#, WPF, Entity Framework
C#	Язык программирования	Современный объектно-ориентированный язык, часть платформы .NET
WPF (Windows Presentation Foundation)	Технология создания интерфейса	Позволяет создавать насыщенные графические интерфейсы на основе XAML
.NET 8.0	Платформа разработки	Кроссплатформенная, производительная, с долгосрочной поддержкой
PostgreSQL 17	Система управления базами данных	Бесплатная, надежная, объектно-реляционная СУБД
EntityFramework Core	ORM (Object-Relational Mapping)	Облегчает взаимодействие с базой данных через объекты C#
Npgsql	Провайдер для PostgreSQL	Обеспечивает подключение к PostgreSQL из .NET
Git	Система контроля версий	Распределенная система управления версиями
MSTest	Фреймворк для модульного тестирования	Встроенный в Visual Studio инструмент для создания тестов
Moq	Библиотека для создания mock-объектов	Позволяет изолировать тестируемые компоненты

Среда программирования

Microsoft Visual Studio 2022 – интегрированная среда разработки (IDE), которая предоставляет comprehensive набор инструментов для создания приложений на платформе .NET.

Основные возможности Visual Studio, использованные в работе:

1. Редактор кода:
   - Подсветка синтаксиса C# и XAML;

- IntelliSense – автодополнение кода, подсказки;
- Быстрая навигация по коду (Go To Definition, Find All References);
- Рефакторинг (переименование, извлечение метода).

2. Визуальный конструктор XAML:
   - Проектирование интерфейса в режиме реального времени;
   - Перетаскивание элементов управления;
   - Настройка свойств через окно Properties.
3. Отладчик:
   - Установка точек остановок;
   - Пошаговое выполнение кода;
   - Просмотр значений переменных;
   - Окно Immediate для выполнения выражений.
4. Работа с базами данных:
   - Подключение к PostgreSQL;
   - Выполнение SQL-запросов;
   - Просмотр структуры БД.
5. Интеграция с Git:
   - Визуальное управление репозиторием;
   - Просмотр изменений;
   - Создание коммитов и веток.
6. Менеджер пакетов NuGet:
   - Установка сторонних библиотек;
   - Обновление пакетов;
   - Управление зависимостями.
7. Тестовый обозреватель:
   - Запуск модульных тестов;
   - Просмотр результатов;
   - Отладка тестов.

Диаграмма классов

Диаграмма классов разработанного приложения представлена на рисунках 1-3. Она отображает архитектуру системы, основные сущности и связи между ними.

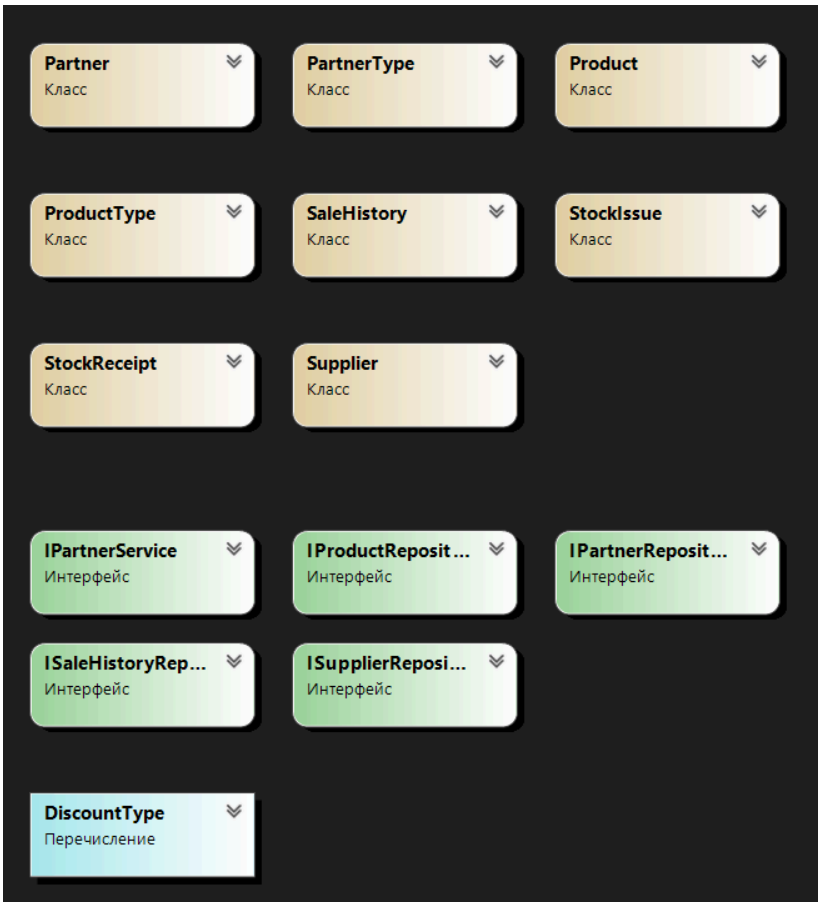

Рисунок 1 - Диаграмма классов слоя Models

Рисунок 2 - Диаграмма классов слоя Data

Рисунок 3 - Диаграмма классов слоя Services

Основные компоненты диаграммы классов:

1. Слой Models (tebenkova_Models):
   - Partner – класс, представляющий партнера (клиента);
   - Product – класс товара;
   - SaleHistory – класс истории продаж;
   - PartnerType – справочник типов партнеров;
   - ProductType – справочник типов товаров;
   - Supplier – класс поставщика;
   - WarehouseItem – класс складской позиции.
2. Интерфейсы репозиториев:
   - IPartnerRepository – методы для работы с партнерами;
   - IProductRepository – методы для работы с товарами;
   - ISaleHistoryRepository – методы для работы с историей продаж;
   - ISupplierRepository – методы для работы с поставщиками.
3. Слой Data (tebenkova_Data):
   - AppDbContext – контекст базы данных Entity Framework;
   - PartnerRepository – реализация репозитория партнеров;
   - ProductRepository – реализация репозитория товаров;
   - SaleHistoryRepository – реализация репозитория истории продаж.
4. Слой Services (tebenkova_Services):
   - PartnerService – бизнес-логика для работы с партнерами;
   - ProductService – бизнес-логика для работы с товарами;

- ReportService – формирование отчетов;
- DiscountHelper – вспомогательный класс для расчета скидок;
- ValidationHelper – вспомогательный класс для валидации данных.

5. DTO (Data Transfer Objects):
   - PartnerDto – объект передачи данных партнера;
   - ProductDto – объект передачи данных товара;
   - SaleHistoryDto – объект передачи данных истории продаж;
   - WarehouseItemDto – объект передачи данных складской позиции.
6. Слой Presentation (tebenkova_wpf):
   - MainWindow – главное окно приложения;
   - PartnerEditWindow – окно редактирования партнера;
   - ProductEditWindow – окно редактирования товара;
   - Converters – конвертеры для привязки данных.

Разработка программного модуля согласно требованиям

Разработка велась в соответствии с техническим заданием (Приложение 1). Основные требования включали:

- Создание базы данных в PostgreSQL, приведенной к 3 нормальной форме;
- Разработку приложения на WPF C# с использованием Entity Framework Core;
- Реализацию функционала просмотра, добавления, редактирования и удаления партнеров;
- Реализацию функционала просмотра, добавления, редактирования и удаления товаров;
- Расчет скидки для партнера на основе истории продаж;
- Отображение складских остатков с цветовой индикацией;
- Формирование отчетов по различным критериям;
- Создание модульных тестов.

Этапы разработки программного обеспечения

На начальном этапе разработки был осуществлен выбор архитектурного шаблона и структуры решения. Проанализировав требования технического задания, было принято решение использовать многослойную архитектуру (N-tier architecture), которая обеспечивает разделение ответственности между компонентами системы.

1. Выбор шаблона проекта:
   - Создано решение tebenkova_Practice.sln;
   - Добавлены проекты: tebenkova_Models (библиотека классов), tebenkova_Data (библиотека классов), tebenkova_Services (библиотека классов), tebenkova_wpf (WPF-приложение), tebenkova_Tests (проект тестов);
   - Установлены необходимые NuGet-пакеты в каждый проект.

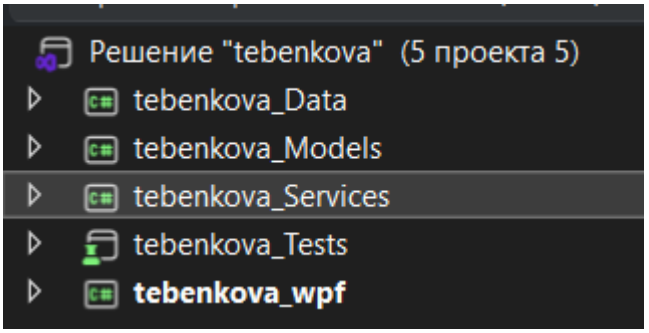

Рисунок 4 - Структура проекта

2. Составление схемы информационной базы:

Проектирование базы данных является ключевым этапом разработки, от которого зависит эффективность работы всей системы. Процесс проектирования включал следующие подэтапы:

1. Анализ предметной области

На основе описания деятельности ООО «Цветы Прикамья» и требований технического задания были выделены следующие бизнес-сущности:

- Партнеры (Partners) – клиенты компании, покупающие продукцию;
- Типы партнеров (PartnerTypes) – справочник организационно-правовых форм (ООО, ИП, ЗАО, АО);

- Товары (Products) – продукция, реализуемая компанией;
- Категории товаров (ProductTypes) – справочник категорий (цветы, горшки, удобрения);
- Поставщики (Suppliers) – организации, поставляющие товары;
- История продаж (SalesHistories) – информация о реализованной продукции партнерам;
- Поступления на склад (StockReceipts) – приход товаров от поставщиков;
- Списания со склада (StockIssues) – отпуск товаров или списание.

2. Логическое проектирование

На этом этапе определены атрибуты каждой сущности и связи между ними:

Сущность «Partner» (Партнер):

- Id (int) – уникальный идентификатор (первичный ключ);
- TypeId (int) – ссылка на тип партнера (внешний ключ);
- Name (string) – наименование организации;
- LegalAddress (string) – юридический адрес;
- INN (string) – ИНН (10 или 12 цифр);
- DirectorName (string) – ФИО директора;
- Phone (string) – контактный телефон;
- Email (string) – электронная почта;
- Rating (int) – рейтинг (от 0 до 10).

Сущность «Product» (Товар):

- Id (int) – уникальный идентификатор;
- Article (string) – артикул (уникальный);
- Name (string) – наименование товара;
- ProductTypeId (int) – ссылка на категорию;
- Description (string) – описание;
- Unit (string) – единица измерения (шт, кг, уп);
- QuantityInStock (decimal) – количество на складе;

- MinStockQuantity (decimal) – минимально допустимый запас;
- CostPrice (decimal) – себестоимость;
- SupplierId (int) – ссылка на поставщика;
- Location (string) – место хранения на складе.

Сущность «SaleHistory» (История продаж):

- Id (int) – уникальный идентификатор;
- PartnerId (int) – ссылка на партнера;
- ProductId (int) – ссылка на товар;
- SaleDate (date) – дата продажи;
- Quantity (int) – количество;
- SalePrice (decimal) – цена продажи.

3. Определение связей между сущностями

   На основе анализа бизнес-логики установлены следующие связи:

Связь	Тип	Описание
PartnerType → Partner	Один-ко-многим	Один тип может быть у многих партнеров
Partner → SaleHistory	Один-ко-многим	Один партнер может иметь много продаж
Product → SaleHistory	Один-ко-многим	Один товар может быть продан много раз
Supplier → Product	Один-ко-многим	Один поставщик поставляет много товаров
Product → StockReceipt	Один-ко-многим	Один товар может много раз поступать
Product → StockIssue	Один-ко-многим	Один товар может много раз списываться

3. Схема базы данных:

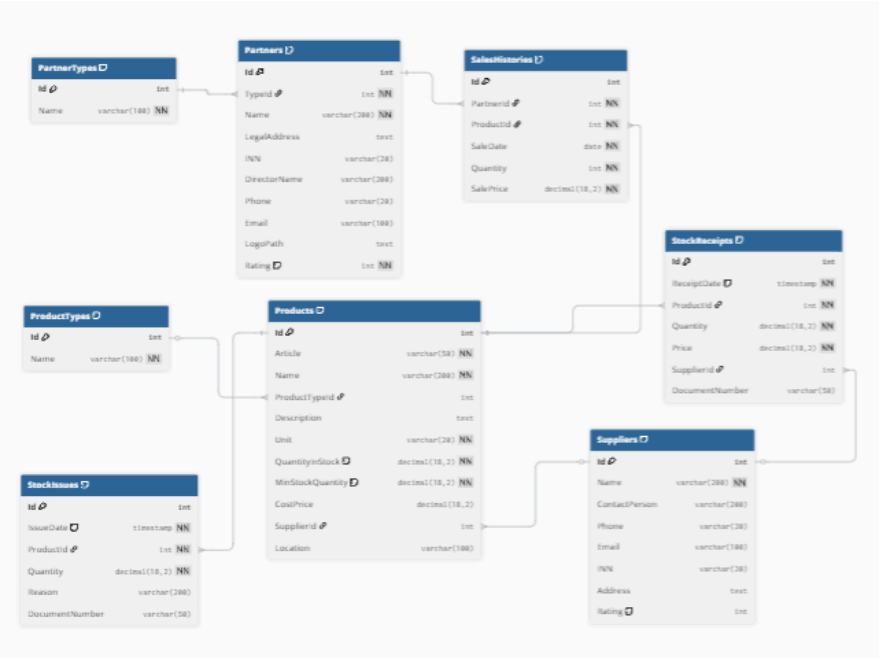

Рисунок 5 - Схема базы данных

4. Документирование системы:
   - Создана техническая документация;
   - Описана архитектура приложения;
   - Составлено руководство пользователя;
   - Подготовлен отчет по практике.
5. Тестирование:
   - Проведено модульное тестирование (unit-тесты);
   - Выполнено интеграционное тестирование;
   - Проведено функциональное тестирование;
   - Исправлены выявленные ошибки.
6. Сопровождение системы:

- Подготовлены инструкции по установке;
- Передано приложение для опытной эксплуатации;
- Получена обратная связь от пользователей.

Разработка форм для ввода, просмотра и редактирования данных

1. Главное окно (MainWindow):

Главное окно содержит вкладки для переключения между разделами:

- Вкладка "Партнеры" – таблица со списком партнеров, панель инструментов (добавление, редактирование, удаление, поиск);
- Вкладка "Товары" – таблица товаров с аналогичным функционалом;
- Вкладка "Склад" – информация о складских остатках с цветовой индикацией статуса;
- Вкладка "Отчеты" – кнопки для формирования различных отчетов.

Основные элементы:

- DataGrid – для отображения данных в табличном виде;
- Кнопки действий (добавить, редактировать, удалить, обновить);
- Поле поиска;
- Строка состояния с информацией о количестве записей.

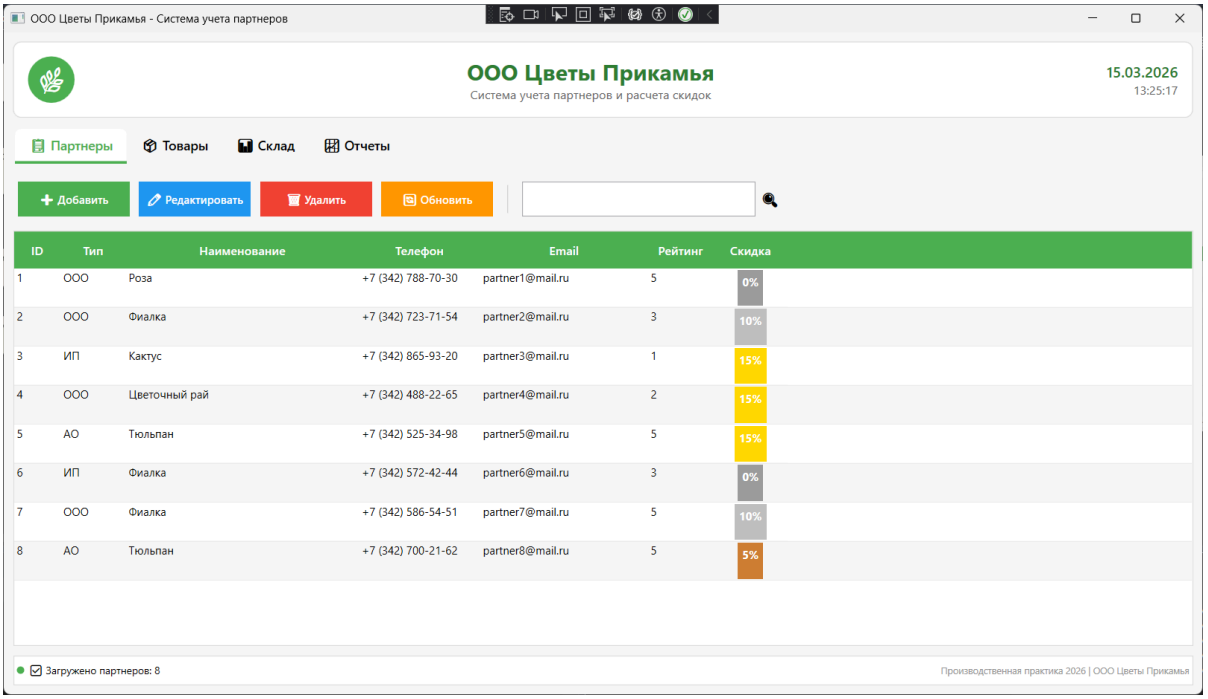

Рисунок 6 - Интерфейс главного окна

2. Окно добавления/редактирования партнера (PartnerEditWindow):

Содержит поля:

- Тип партнера (выпадающий список: ООО, ИП, ЗАО, АО);
- Наименование организации;
- Юридический адрес;
- ИНН;
- ФИО директора;
- Телефон;

- Email;
- Рейтинг (число от 0 до 10).

Реализована валидация вводимых данных:

- Проверка заполнения обязательных полей;
- Проверка корректности ИНН (10 или 12 цифр);
- Проверка диапазона рейтинга.

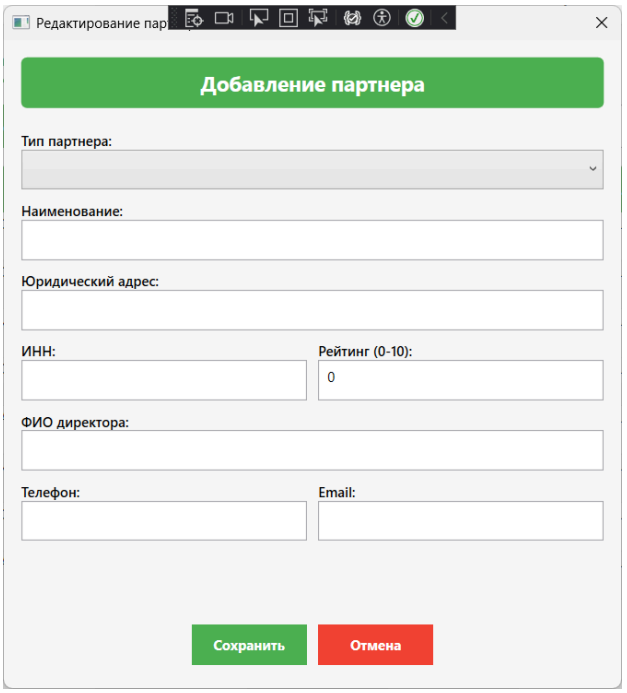

Рисунок 7 - Интерфейс окна добавления партнера

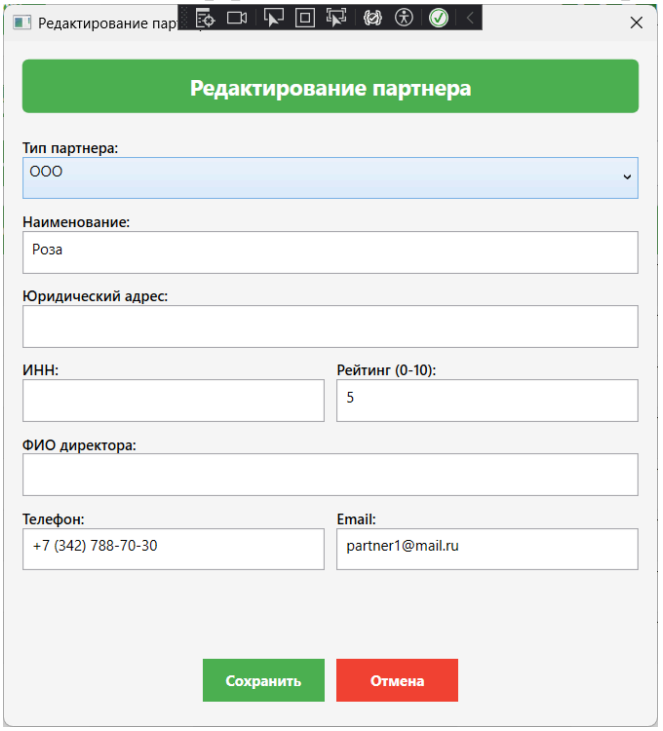

Рисунок 8 - Интерфейс окна редактирования партнера

3. Окно добавления/редактирования товара (ProductEditWindow):

Содержит поля:

- Артикул;
- Наименование товара;
- Категория (выпадающий список);
- Цена;
- Количество на складе.

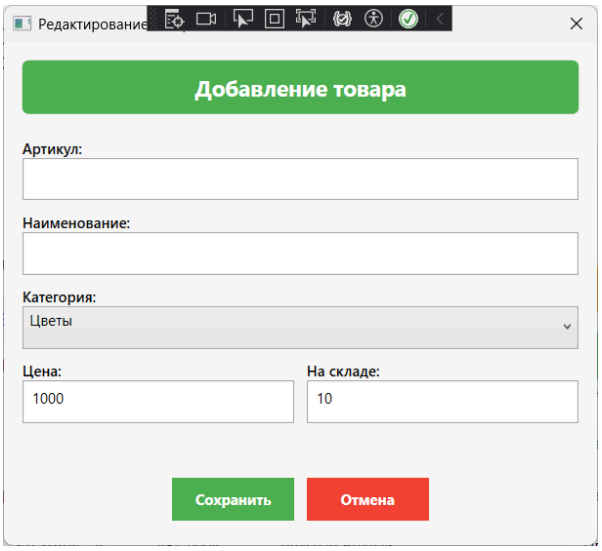

Рисунок 9 - Интерфейс окна добавления товара

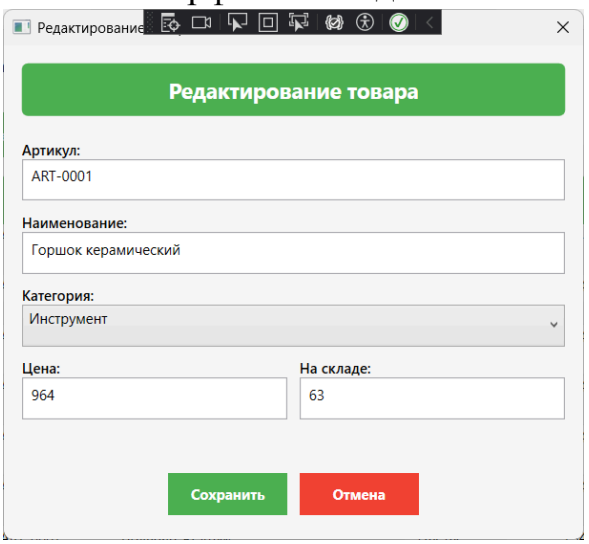

Рисунок 10 - Интерфейс окна редактирования товара

Этапы разработки интерфейса

1. Проектирование:

- Определена структура главного окна (вкладки для разных разделов);
- Спроектирован состав таблиц для отображения данных;
- Определен набор полей для форм ввода;

- Разработана навигация между окнами.

2. Прототипирование:

- Созданы низкодетализированные макеты на бумаге;
- Реализован первый работающий прототип для демонстрации руководителю;
- Внесены коррективы по результатам обсуждения.

3. Стилизация:

- Выбрана цветовая гамма в корпоративных цветах (зеленый, белый, серый);
- Разработаны стили для кнопок, таблиц, полей ввода;

Реализована цветовая индикация:

- Скидки: серый (0%), бронзовый (5%), серебряный (10%), золотой (15%);
- Статусы склада: зеленый (норма), оранжевый (требуется закуп), красный (отсутствует);
    - Добавлены иконки для кнопок (➕, ✏️, 🗑️, 🔄, 🔍);
    - Настроены скругленные углы для элементов.

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества

В процессе разработки программного обеспечения для ООО «Цветы Прикамья» применялся комплекс методов и подходов, направленных на получение качественного кода, полностью соответствующего функциональным требованиям.

Объектно-ориентированное программирование (ООП) позволило создать гибкую архитектуру. Инкапсуляция обеспечила сокрытие внутренней реализации классов, наследование позволило избежать дублирования кода, а полиморфизм дал возможность легко подменять реализации (например, в тестах используются mock-объекты вместо реальных репозиториев).

Принципы SOLID обеспечили создание кода, который легко поддерживать и расширять. Каждый класс отвечает только за одну задачу,

зависимости строятся на абстракциях, а интерфейсы разделены по функциональному назначению, что делает систему гибкой и тестируемой.

Многослойная архитектура разделила приложение на независимые уровни: слой представления (WPF), слой бизнес-логики (сервисы), слой доступа к данным (репозитории) и слой моделей. Такое разделение позволяет тестировать каждый слой отдельно и при необходимости заменять один слой без изменения других.

Паттерны проектирования помогли решить типовые задачи проверенными способами. Repository инкапсулирует логику доступа к данным, Dependency Injection обеспечивает слабую связанность компонентов, а DTO передают только необходимые данные между слоями.

Принципы качественного кода включали DRY (избегание дублирования через вынесение общего кода в отдельные методы и классы-хелперы), KISS (простота решений) и YAGNI (отказ от избыточной функциональности). Использовались осмысленные имена переменных и методов, сложные участки снабжались комментариями.

Обработка исключений реализована через блоки try-catch для перехвата ошибок с последующим логированием и информированием пользователя. Валидация входных данных предотвращает некорректные операции, а null-условные операторы защищают от NullReferenceException.

Стандарты кодирования соблюдались через единый стиль именования (PascalCase для классов и методов, camelCase для переменных), правильное форматирование и использование XML-комментариев для публичных API.

Автоматизированная проверка осуществлялась средствами Visual Studio: анализатор кода подсвечивал проблемы, Roslyn Analyzers проверяли соблюдение правил, а отладчик позволял отслеживать выполнение в реальном времени.

Применение всех этих методов позволило создать надежное, поддерживаемое и полностью соответствующее требованиям приложение, готовое к длительной эксплуатации.

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

В процессе разработки проводилось регулярное инспектирование кода для проверки его соответствия установленным стандартам и обеспечения высокого качества программного продукта.

Виды инспекции

В ходе практики применялись следующие виды инспекции кода:

- Самопроверка выполнялась после каждого значимого этапа разработки. Я самостоятельно просматривала написанный код, проверяя соблюдение стандартов оформления, наличие комментариев к сложным участкам и отсутствие очевидных ошибок.
- Формальная инспекция проводилась руководителем практики от организации по завершении разработки каждого модуля. Руководитель проверял код на соответствие требованиям технического задания и стандартам кодирования, выявленные замечания оперативно устранялись.
- Автоматизированная инспекция осуществлялась непрерывно средствами Visual Studio, которые подсвечивают синтаксические ошибки и потенциальные проблемы прямо в процессе написания кода.

Актуальные стандарты кодирования

При разработке использовались следующие стандарты и соглашения:

- Общепринятые соглашения по написанию кода на C# включают правила именования: классы и публичные методы – PascalCase, локальные переменные – camelCase, приватные поля – _camelCase, интерфейсы – с префиксом I. Все имена должны быть осмысленными и отражать назначение элементов.
- Правила форматирования предписывают использовать отступы в 4 пробела, открывать фигурные скобки на новой строке, оставлять пустые строки между логическими блоками.

- Требования к комментированию включают XML-комментарии для публичных методов и классов с описанием назначения, параметров и возвращаемых значений. Сложные алгоритмы снабжаются пояснительными комментариями.
- Принципы обработки исключений требуют использования блоков try-catch для перехвата ошибок и информирования пользователя.
- ГОСТ 19.701-90 использовался при проектировании алгоритмов и построении блок-схем.

Программы для инспекции

Для проведения инспекции использовались следующие средства:

- Visual Studio 2022 обеспечивает непрерывную проверку кода в реальном времени, подсвечивая синтаксические ошибки и потенциальные проблемы. Встроенные средства рефакторинга позволяют улучшать структуру кода.
- Roslyn Analyzers выполняют статический анализ кода, проверяя соблюдение правил оформления и выявляя потенциальные ошибки.
- IntelliSense помогает писать корректный код, предлагая автодополнение и подсказки.
- Отладчик Visual Studio используется для проверки корректности работы кода в динамике.
- В результате регулярных инспекций все выявленные замечания были устранены, а итоговый код приведен в соответствие с установленными стандартами кодирования.

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств

В процессе разработки проводилась отладка созданных модулей с целью выявления и устранения ошибок с использованием встроенных средств Visual Studio 2022.

Процесс проведения отладки ПО

Отладка осуществлялась поэтапно и включала следующие шаги:

- Подготовка к отладке. Проект собирался в конфигурации Debug, содержащей отладочную информацию. Определялись проблемные участки и воспроизводились условия возникновения ошибок.
- Установка точек остановок. В ключевых местах кода (методы загрузки данных, алгоритмы расчета скидок, обработчики событий) устанавливались точки останова, на которых выполнение программы приостанавливалось для анализа.
- Пошаговое выполнение кода. Использовались режимы Step Into (F11) для захода в вызываемые методы, Step Over (F10) для выполнения текущей строки без захода в методы и Step Out (Shift+F11) для выхода из текущего метода.
- Просмотр значений переменных. В окнах Autos, Locals и Watch отслеживались значения переменных в процессе выполнения, что позволяло выявить некорректные данные. Окно Immediate использовалось для выполнения выражений в реальном времени.
- Анализ стека вызовов. При возникновении ошибок окно Call Stack показывало последовательность вызовов методов, помогая определить источник проблемы.
- Исправление ошибок. После выявления причины ошибки в код вносились исправления, затем отладка повторялась для проверки правильности исправления.

Основные выявленные ошибки

В ходе отладки были обнаружены и устранены следующие ошибки:

- Ошибка NullReferenceException возникала при обращении к неинициализированным объектам. Исправлено добавлением проверок на null.
- Ошибки в логике расчета скидок выявлялись при неверных условиях. Условные операторы скорректированы для правильного расчета.

- Ошибки при работе с базой данных проявлялись из-за несоответствия типов. Параметры запросов приведены в соответствие со структурой БД.
- Ошибки обновления интерфейса после добавления записей исправлены добавлением вызовов методов обновления списков.

Инструменты отладки

- Основным инструментом выступал встроенный отладчик Visual Studio 2022 с возможностями установки точек останова, пошагового выполнения, просмотра переменных и анализа стека вызовов.

В результате отладки все выявленные ошибки устранены, приложение работает стабильно и корректно обрабатывает все сценарии использования.

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения

В процессе разработки программного обеспечения для ООО «Цветы Прикамья» проводилось тестирование созданных модулей с целью проверки их работоспособности и соответствия функциональным требованиям. Тестирование включало разработку тестовых наборов, сценариев и создание модульных тестов.

Процесс проведения тестирования ПО

Тестирование проводилось на нескольких уровнях и включало следующие этапы:

- Модульное тестирование выполнялось для проверки отдельных компонентов в изоляции от остальной системы. Для этого использовался фреймворк MSTest, а для изоляции зависимостей применялась библиотека Moq, позволяющая создавать mock-объекты репозиториев. Модульные тесты охватывали ключевые методы бизнес-логики: расчет скидок, валидацию данных, работу сервисов.
- Интеграционное тестирование проверяло взаимодействие между модулями, например работу сервисов с репозиториями. Для этого

использовалась тестовая база данных в памяти (InMemory), что позволяло проводить тесты без влияния на реальные данные.

- Функциональное тестирование проверяло соответствие системы требованиям технического задания. Выполнялись все пользовательские сценарии: добавление, редактирование, удаление и поиск записей, расчет скидок, формирование отчетов. Результаты фиксировались в чек-листах.
- UI-тестирование проводилось вручную и проверяло корректность отображения данных, работу кнопок, навигацию между вкладками и цветовую индикацию статусов.

Разработка тестовых наборов и сценариев

На основе функциональных требований были разработаны тестовые сценарии, охватывающие все ключевые возможности системы. Для каждого сценария определены входные данные, ожидаемый результат и условия выполнения.

Создание unit-тестов

В ходе работы было создано 57 модульных тестов, охватывающих основные компоненты приложения. Структура тестов организована в проекте tebenkova_Tests следующим образом:

- Тестирование вспомогательных классов (Helpers) — 26 тестов:
    - DiscountHelperTests содержит 9 тестов для проверки расчета скидок при различных суммах продаж. Тесты проверяют граничные условия: суммы менее 10000 рублей (скидка 0%), от 10000 до 50000 рублей (скидка 5%), от 50000 до 300000 рублей (скидка 10%), более 300000 рублей (скидка 15%), а также обработку пустого списка продаж.
    - ValidationHelperTests включает 15 тестов для проверки валидации данных: корректность ИНН (10 и 12 цифр), проверка телефона (допустимые символы), проверка email-адресов, а также тесты для КПП, ОГРН и банковских реквизитов.

    - ExportHelperTests содержит 2 теста для проверки экспорта данных в CSV-формат и форматирования дат и чисел.
- Тестирование репозиториев (Repositories) — 6 тестов:
    - PartnerRepositoryTests включает 6 тестов для проверки операций с базой данных: получение всех записей, получение по идентификатору, добавление, обновление, удаление и проверка существования записи. Для тестирования использовалась InMemory-база данных.
- Тестирование сервисов (Services) — 25 тестов:
    - PartnerServiceTests содержит 13 тестов, проверяющих бизнес-логику работы с партнерами: получение списка, получение по идентификатору, создание нового партнера с проверкой валидации, обновление, удаление, расчет скидки, поиск по различным критериям.
    - ProductServiceTests включает 5 тестов для проверки операций с товарами: создание, обновление, удаление, получение списка товаров с низким запасом и формирование сводки по складу.
    - SupplierServiceTests содержит 5 тестов для проверки работы с поставщиками: создание, обновление, удаление (с проверкой наличия связанных товаров), получение статистики по поставщику.
    - ReportServiceTests включает 2 теста для проверки формирования отчетов: сводка по дашборду и отчет по продажам за период.

Результаты тестирования

Все 57 тестов были успешно выполнены. В ходе тестирования были выявлены и устранены следующие недостатки:

- Исправлена логика расчета скидки для граничных значений сумм;
- Добавлены проверки на null в методах сервисов;
- Улучшена валидация вводимых данных.

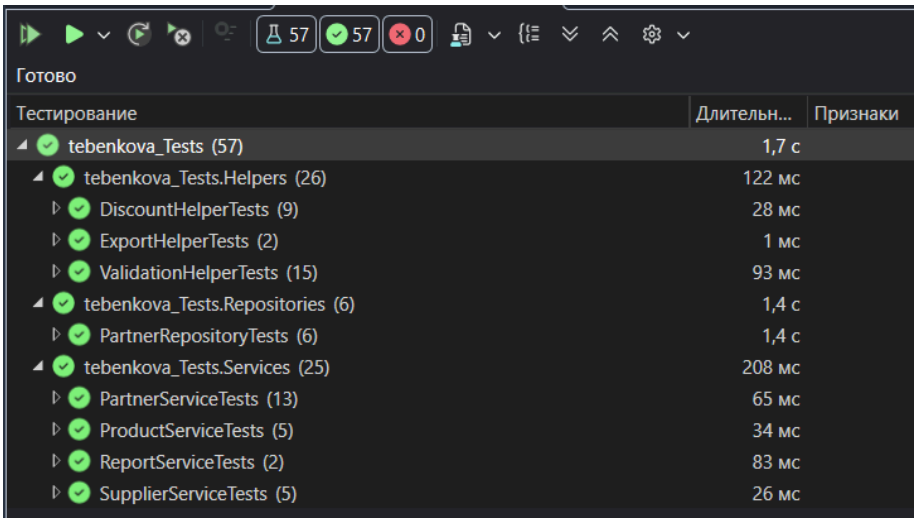

Рисунок 11 - Результаты выполнения тестов

3.1.7. Интегрирование программных модулей

В процессе разработки программного обеспечения для ООО «Цветы Прикамья» выполнялась интеграция отдельных модулей в единую работающую систему. Интеграция проводилась поэтапно с использованием восходящей стратегии, когда сначала интегрировались модули нижних уровней, а затем – верхних.

Процесс интеграции программных модулей

Интеграция модулей осуществлялась в несколько этапов:

Этап 1. Интеграция уровня данных. На данном этапе проверялась работа репозиториев с базой данных PostgreSQL. Был создан и настроен контекст базы данных AppDbContext, установлено подключение к серверу, проверены CRUD-операции для каждой сущности. Тестировались связи между таблицами и каскадные операции. Все репозитории успешно взаимодействовали с базой данных, корректно выполняли запросы и возвращали ожидаемые результаты.

Этап 2. Интеграция уровня бизнес-логики. Сервисы (PartnerService, ProductService, ReportService) были подключены к соответствующим репозиториям через внедрение зависимостей в конструкторы. Проверялась корректность бизнес-правил: расчет скидок, валидация данных, формирование отчетов. Для проверки использовались как реальные, так и

тестовые данные. Все сервисы успешно взаимодействовали с репозиториями и возвращали корректные результаты.

Этап 3. Интеграция пользовательского интерфейса. На этом этапе окна WPF были подключены к сервисам. В главном окне создавались экземпляры сервисов, которые затем использовались для загрузки и отображения данных. Обработчики событий кнопок вызывали соответствующие методы сервисов. Проверялась привязка данных (data binding) и обновление интерфейса при изменениях.

Этап 4. Интеграция всех модулей в единое приложение. После интеграции каждого уровня отдельно выполнялась проверка взаимодействия всех компонентов в составе единого приложения. Тестировались:

- Переключение между вкладками и загрузка соответствующих данных;
- Передача данных между модулями (например, идентификатора выбранного партнера в окно редактирования);
- Совместная работа модулей при формировании отчетов (сбор данных из разных репозиториев).

Архитектура интеграции

Интеграция модулей построена по следующей схеме:

- Пользовательский интерфейс (окна WPF) обращается к сервисам;
- Сервисы реализуют бизнес-логику и обращаются к репозиториям;
- Репозитории выполняют операции с базой данных через Entity Framework Core;
- Данные передаются между слоями с помощью DTO-объектов.

Способы интеграции

Для интеграции модулей использовались следующие механизмы:

- Прямые вызовы методов применялись для передачи управления между слоями: методы сервисов вызывались из UI, методы репозиториев – из сервисов.
- Внедрение зависимостей (Dependency Injection) обеспечивало слабую связанность компонентов: сервисы получали репозитории через

конструкторы, что позволяло легко подменять реализации при тестировании.

- Событийная модель использовалась для уведомления об изменениях данных: после добавления или редактирования записей вызывалось обновление списков в интерфейсе.

Проблемы, возникшие при интеграции

В процессе интеграции были выявлены и устранены следующие проблемы:

- Несоответствие типов данных между слоями решено путем создания DTO-объектов, которые содержат только необходимые для передачи данные и не создают циклических ссылок.
- Потеря данных при передаче устранена добавлением маппинга между сущностями и DTO с помощью специальных методов расширения.
- Проблемы с обновлением интерфейса после изменения данных решены путем явного вызова методов перезагрузки списков после завершения операций.

3.1.8. Верификация и аттестация программного обеспечения

После завершения разработки и интеграции всех модулей была проведена верификация и аттестация программного обеспечения для подтверждения его соответствия требованиям и готовности к эксплуатации.

Процесс верификации программного обеспечения

Верификация – процесс проверки того, что программное обеспечение разработано в соответствии с заданными требованиями и спецификациями.

Верификация проводилась по следующим направлениям:

- Проверка соответствия техническому заданию. Было выполнено сравнение реализованного функционала с требованиями технического задания. Проверены все обязательные функции: управление партнерами (добавление, редактирование, удаление, поиск), управление товарами, учет складских остатков с цветовой индикацией, расчет

скидок на основе истории продаж, формирование отчетов. Все требования выполнены в полном объеме.

- Анализ кода (Code Review). Проведена проверка исходного кода на соответствие стандартам кодирования: правильность именования переменных и методов, наличие комментариев, форматирование кода, соблюдение принципов SOLID. Все замечания, выявленные в ходе инспекций, были устранены.
- Проверка обработки ошибок. Протестировано поведение приложения при различных ошибочных ситуациях: ввод некорректных данных (пустые поля, неверный формат ИНН, выход рейтинга за допустимые пределы), отсутствие подключения к базе данных, попытка удаления связанных записей. Приложение корректно обрабатывает все ошибки, выводит информативные сообщения и не завершается аварийно.
- Проверка производительности. Выполнено тестирование с большим объемом данных (до 1000 записей). Загрузка списков занимает не более 2-3 секунд, поиск выполняется мгновенно, интерфейс остается отзывчивым.

Процесс аттестации программного обеспечения

Аттестация – подтверждение того, что программное обеспечение готово к эксплуатации и соответствует потребностям пользователя.

Аттестация включала следующие этапы:

Приемо-сдаточные испытания. Испытания проводились в присутствии руководителя практики от организации. Были выполнены все тестовые сценарии, предусмотренные программой испытаний:

№	Тестовый сценарий	Результат
1	Запуск приложения, проверка загрузки главного экрана	Успешно
2	Просмотр списка партнеров, проверка отображения данных	Успешно
3	Добавление нового партнера с корректными данными	Успешно

4	Добавление партнера с ошибками (пустое наименование)	Успешно (появилось сообщение)
5	Редактирование существующего партнера	Успешно
6	Удаление партнера с подтверждением	Успешно
7	Поиск партнера по наименованию и телефону	Успешно
8	Переход на вкладку “Товары”, проверка списка	Успешно
9	Добавление нового товара	Успешно
10	Переход на вкладку "Склад", проверка цветовой индикации	Успешно
11	Формирование отчетов по разным типам	Успешно

Оценка удобства использования. Руководителем практики была проведена оценка эргономичности интерфейса. Отмечено, что интерфейс интуитивно понятен, все функции доступны с главного окна, цветовая индикация помогает быстро оценить ситуацию, поиск работает быстро и точно.

Составление акта аттестации. По результатам испытаний составлен акт, в котором зафиксировано:

- Программное обеспечение полностью соответствует техническому заданию;
- Все функции работают корректно, критических ошибок не выявлено;
- Интерфейс удобен для использования, не требует длительного обучения;
- Программное обеспечение рекомендовано к опытной эксплуатации в ООО «Цветы Прикамья».

Передача в опытную эксплуатацию

После успешной аттестации программное обеспечение было установлено на рабочем месте менеджера отдела продаж. Проведен инструктаж пользователя по работе с системой, передана пользовательская документация. Назначен срок опытной эксплуатации – 2 недели, после чего будет принято решение о постоянном использовании системы.

Заключение по верификации и аттестации

В результате проведенной верификации и аттестации подтверждено, что разработанное программное обеспечение:

- полностью соответствует требованиям технического задания;
- не содержит критических ошибок;
- обеспечивает требуемую функциональность;
- удобно в использовании;
- готово к эксплуатации в условиях реальной деятельности ООО «Цветы Прикамья».

Заключение

В результате прохождения производственной практики с 09 по 17 марта 2026 года были закреплены теоретические знания и расширены профессиональные умения в области разработки программного обеспечения.

По итогам практики получены следующие результаты:

1. Разработана полнофункциональная информационная система для автоматизации учета партнеров, товаров и складских запасов ООО «Цветы Прикамья», включающая модули управления партнерами, товарами, складского учета и формирования отчетности.
2. Спроектирована и создана база данных в PostgreSQL, приведенная к третьей нормальной форме, с обеспечением ссылочной целостности и оптимизированной структурой таблиц.
3. Реализовано многослойное приложение на платформе .NET с использованием технологий WPF и Entity Framework Core, обеспечивающее:
   - ведение справочников партнеров и товаров с возможностью добавления, редактирования и удаления записей;
   - учет складских остатков с автоматической цветовой индикацией статусов (норма, требуется закуп, отсутствует);
   - расчет скидок для партнеров на основе истории продаж по заданным правилам;
   - поиск и фильтрацию данных по различным критериям;
   - формирование пяти видов аналитических отчетов.
4. Разработано 57 модульных тестов, охватывающих ключевые компоненты системы (хелперы, репозитории, сервисы), что подтверждает корректность работы бизнес-логики.
5. Выполнена интеграция всех модулей в единое приложение с использованием восходящей стратегии и внедрения зависимостей.

6. Проведены отладка и всестороннее тестирование программного обеспечения, включая модульное, интеграционное, функциональное и UI-тестирование.
7. Осуществлена верификация и аттестация программного продукта, подтвердившая его полное соответствие требованиям технического задания.

По окончании практики получен практический опыт:

- интеграции модулей в программное обеспечение;
- отладки программного модуля с использованием специализированных программных средств;
- разработки тестовых наборов и тестовых сценариев для программного обеспечения;
- инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

Разработанное программное обеспечение передано в опытную эксплуатацию в ООО «Цветы Прикамья», получена положительная оценка руководителя практики. Все поставленные задачи выполнены в полном объеме, что подтверждает успешное прохождение производственной практики.

Список используемых источников

1. ГОСТ Р ИСО/МЭК 9294-93. Информационная технология. Руководство по управлению документированием программного обеспечения.

2. ГОСТ Р ИСО/МЭК 9126-93. Информационная технология. Оценка программной продукции. характеристика качества и руководство по их применению.

3. ГОСТ Р ИСО/МЭК 9127-94. Системы обработки информации. Документация пользователя и информация на упаковке для потребительских программных пакетов.

4. ГОСТ Р ИСО/МЭК 8631-94. Информационная технология. Программные конструктивы и условные обозначения для их представления.

5. ГОСТ Р ИСО/МЭК 12119:2000. Информационная технология. Пакеты программных средств. Требования к качеству и испытания.

6. ГОСТ 2.108 - 68 - Спецификация (ЕСКД)

7. ГОСТ 2.105 - 95 - Общие требование к текстовым документам. (ЕСКД)

8. ГОСТ 2.103 - 68 - Стадии разработки (ЕСКД)

9. ГОСТ 2.106 - 68 - Ведомость спецификаций (ЕСКД)

10. ГОСТ - 7.1- 84 - Библиографическое описание документа. Общие требования и правила составления. (ЕСКД)

11. ГОСТ 2.004 - 88 - Общие требования к выполнению конструкторских и технологических документов на печатающих и графических устройствах вывода ЭВМ. (ЕСКД)

12. ГОСТ 19.101-77 - Виды программ и программных документов

13. ГОСТ 19.103-77 - Обозначение программ и программных документов.

14. ГОСТ 19.102 - 77 - Стадии разработки

15. ГОСТ 19.104 - 78 - Основные надписи

16. ГОСТ 19.105 - 78 - Общие требования к программным документам

17. ГОСТ 19.201 - 78 - Техническое задание.

18. ГОСТ 19.202 - 78 - Спецификация 19. ГОСТ

19.781- 90 - Термины и определения

20. ГОСТ 19.701-90 - Схемы алгоритмов, программ данных и схем. Условные обозначения и правила выполнения

21. ГОСТ Р ИСО/МЭК 12207-2010 «Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств»,

22. Федорова Г. Н. Участие в интеграции программных модулей. Учебное пособие / Федорова Г.Н. – М.Ж Издательский дом «Академия», 2016 – 304с.

23. Федорова Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности: Учебное пособие. / Федорова Г.Н. - М.:КУРС, НИЦ ИНФРАМ, 2016. - 336 с. http://znanium.com/catalog.php?bookinfo=544732

24. Оценка качества программного обеспечения: Практикум: Учебное пособие / Б.В. Черников, Б.Е. Поклонов; Под ред. Б.В. Черникова - М.: ИД ФОРУМ: НИЦ Инфра-М, 2012. - 400 с. http://znanium.com/bookread2.php?book=315269

25. Гуриков С. Р. Введение в программирование на языке Visual C#: Учебное пособие / С.Р. Гуриков. - М.: Форум: НИЦ ИНФРА-М, 2013. - 448 с. http://znanium.com/bookread2.php?book=232661

26. Исаченко О.В. Программное обеспечение компьютерных сетей : учебное пособие. М. : ИНФРА-М, 2013. - 117 с. - (СПО).

27. Стандартизация, сертификация и управление качеством программного обеспечения: Учебное пособие / Ананьева Т.Н., Новикова Н.Г., Исаев Г.Н. - М.:НИЦ ИНФРА-М, 2016. - 232с http://znanium.com/bookread2.php?book=541003

28. Гагарина Л.Г. Технология разработки программного обеспечения: Учебное пособие / Л.Г. Гагарина, Б.Д. Виснадул, Е.В. Кокорева. М.: ООО

"Научно-издательский центр ИНФРА-М", 2013. 400 с. URL: http :// znanium. com / catalog. php ? bookinfo =389963 10

29. Немцова Т. И. Программирование на языке высокого уровня. Программир. на языке С++[Электронный ресурс]: Уч. пос. / Т.И.Немцова и др.; Под ред. Л.Г.Гагариной. М.: ИД ФОРУМ: ИНФРА-М, 2012. 512 с. URL: http://znanium.com/catalog.php?bookinfo=244875; 30. Рудаков А.В. Технология разработки программных продуктов: Учеб.пособие для студ. сред. проф. образования. М.: Издательство «Академия», 2010. 208 с.

31. Черников Б.В. Управление качеством программного обеспечения [Электронный ресурс]: Учебник / Б.В. Черников. - М.: ИД ФОРУМ: ИНФРА-М, 2012. - 240 с.. URL: http://znanium.com/catalog.php?bookinfo=256901.

32. Эйдлина Г.М. Delpfi: программирование в примерах и задачах[Электронный ресурс]: Практикум / Г.М. Эйдлина, К.А. Милорадов. М.: ИЦ РИОР: НИЦ Инфра-М, 2012. 116 с. URL: http :// znanium. com / catalog. php ? bookinfo =319046

Дополнительные источники:

1. Гагарина Л.Г. Разработка и эксплуатация автоматизированных информационных систем [Электронный ресурс]: Учебное пособие / Л.Г. Гагарина. М.: ИД ФОРУМ: НИЦ Инфра-М, 2013.384с. URL: http://znanium.com/catalog.php?bookinfo=368454;

2. Голицына О. Л. Программное обеспечение[Электронный ресурс]: Учебное пособие / О.Л. Голицына, Т.Л. Партыка, И.И. Попов. - 3-е изд., перераб.и доп. М.: Форум, 2010.448с. URL: http://znanium.com/catalog.php?bookinfo=201030;

3. Роббинс, Д. Отладка Windows-приложений [Электронный ресурс] / Д. Роббинс, пер. с англ. М.: ДМК Пресс, 2009. 448 с., ил. URL: http://znanium.com/catalog.php?bookinfo =407747

Интернет-ресурсы:

1. Технология программирования: [Электронныйресурс]. URL: http :// glossary. ru / cgi - bin / gl _ sch 2. cgi ? RSl) turujoo ! vwujwgssowuigto 9

2. Технология программирования:[Электронный ресурс].URL: http ://2 programmer. ru/l1?start=3

3. Информатика:[Электронный ресурс]. URL: http:// klgtu. ru / students / literature / inf _ asu /600. html

4. Языки программирования: [Электронный ресурс]. URL: http://maksakov-sa.ru/TehProgram/index.html

5. Информатика: [Электронный ресурс]. URL: http://object.newmail.ru/obj1.html

6. Delphi и базы данных: [Электронный ресурс]. URL: http://snkey.net/books/delphi/ch4- 2.html

7. Базы данных: [Электронный ресурс]. URL: http://www.sdteam.com/5/.

Приложения

Приложение 1

Техническое задание на разработку программного обеспечения
«Система учета партнеров и складских запасов для ООО "Цветы Прикамья"»

1. Введение

Настоящее техническое задание распространяется на разработку программного обеспечения «Система учета партнеров и складских запасов» для ООО «Цветы Прикамья». Система предназначена для автоматизации процессов учета партнеров, товаров, складских операций и формирования отчетности.

2. Основание для разработки

Основанием для разработки является задание на производственную практику ПП.02.01 Технология разработки программного обеспечения по специальности 09.02.07 «Информационные системы и программирование».

3. Назначение разработки

Система предназначена для автоматизации следующих бизнес-процессов:

- ведение справочника партнеров (клиентов) компании;
- ведение справочника товаров и продукции;
- учет складских остатков с контролем минимального запаса;
- расчет скидок для партнеров на основе истории продаж;
- формирование аналитических отчетов.

4. Требования к программному обеспечению

4.1. Требования к функциональным характеристикам

Система должна обеспечивать выполнение следующих функций:

4.1.1. Управление партнерами:

- просмотр списка партнеров в табличном виде;
- добавление нового партнера с указанием: типа партнера (ООО, ИП, ЗАО, АО), наименования, юридического адреса, ИНН, ФИО директора, контактного телефона, email, рейтинга (целое число от 0 до 10);
- редактирование данных существующего партнера;
- удаление партнера (с подтверждением операции);
- поиск партнеров по наименованию, телефону и email;
- просмотр истории продаж по каждому партнеру.

4.1.2. Управление товарами:

- просмотр списка товаров в табличном виде;
- добавление нового товара с указанием: артикула, наименования, категории, цены, количества на складе;
- редактирование данных существующего товара;
- удаление товара (с подтверждением операции);
- поиск товаров по наименованию и артикулу.

4.1.3. Складской учет:

- отображение текущих остатков товаров на складе;
- контроль минимального запаса по каждому товару;
- автоматическая цветовая индикация статусов:
  - зеленый – норма (остаток выше минимального);
  - оранжевый – требуется закуп (остаток ниже минимального);
  - красный – отсутствует (остаток равен 0);
- отображение сводной информации: общее количество товаров, количество позиций, требующих закуп, количество отсутствующих позиций.

4.1.4. Расчет скидок:

- автоматический расчет скидки для партнера на основе общей суммы его продаж;
- правила расчета скидки:
  - до 10 000 руб. – скидка 0%;

- от 10 000 до 50 000 руб. – скидка 5%;
- от 50 000 до 300 000 руб. – скидка 10%;
- более 300 000 руб. – скидка 15%;

- отображение скидки в процентах с цветовой индикацией (серый, бронзовый, серебряный, золотой).

4.1.5. Формирование отчетов:

- отчет по продажам за выбранный период;
- отчет по товарам (общее количество, общая стоимость);
- финансовый отчет (выручка, расчетная прибыль);
- отчет по партнерам (количество партнеров, средняя скидка);
- складской отчет (позиции, требующие закуп).

4.2. Требования к интерфейсу

- Главное окно должно содержать вкладки для переключения между разделами (Партнеры, Товары, Склад, Отчеты);
- Каждый раздел должен содержать панель инструментов с кнопками основных операций;
- Данные должны отображаться в табличном виде с возможностью сортировки по колонкам;
- Формы ввода должны содержать поля для всех необходимых атрибутов и обеспечивать валидацию вводимых данных;
- Интерфейс должен быть интуитивно понятным, выполненным в едином цветовом стиле;
- Должна отображаться строка состояния с информацией о текущих действиях и количестве записей.

4.3. Требования к архитектуре

Система должна быть реализована с использованием многослойной архитектуры:

- слой моделей (сущности, интерфейсы);
- слой доступа к данным (репозитории, контекст БД);
- слой бизнес-логики (сервисы, DTO, хелперы);
- слой представления (пользовательский интерфейс).

4.4. Требования к базе данных

- Использование СУБД PostgreSQL;
- База данных должна быть приведена к третьей нормальной форме;
- Обеспечение ссылочной целостности с помощью первичных и внешних ключей;
- Создание необходимых индексов для оптимизации поиска.

4.5. Требования к тестированию

- Разработка модульных тестов для ключевых компонентов (расчет скидок, валидация данных, сервисы);
- Покрытие тестами должно составлять не менее 80% бизнес-логики.

5. Состав и содержание работ по этапам

Этап	Содержание работ	Сроки
1	Анализ предметной области, проектирование базы данных	09.03.2026
2	Создание структуры решения, разработка моделей и репозиториев	10.03.2026
3	Разработка сервисов и бизнес-логики	11.03.2026
4	Разработка пользовательского интерфейса (модуль партнеров)	12.03.2026

5	Разработка модулей товаров и склада	13.03.2026
6	Разработка отчетов, написание тестов	14.03.2026
7	Интеграция модулей, отладка, тестирование	15.03.2026
8	Подготовка документации, сдача проекта	16.03.2026

6. Требования к программной документации

В составе разработанного ПО должна быть представлена следующая документация:
- техническое задание (настоящий документ);
- руководство пользователя;
- руководство администратора;
- исходный код приложения.

7. Порядок контроля и приемки

Приемка разработанного программного обеспечения осуществляется руководителем практики от организации. В ходе приемки проверяется:
- соответствие функциональных возможностей требованиям технического задания;
- корректность работы всех модулей;
- отсутствие критических ошибок;
- удобство пользовательского интерфейса.

8. Требования к составу и параметрам технических средств
- Процессор: не ниже Intel Core i3 или аналогичный;
- Оперативная память: не менее 4 ГБ;
- Свободное дисковое пространство: не менее 1 ГБ;
- Операционная система: Windows 10/11;
- Монитор с разрешением не менее 1366×768.

9. Требования к программному обеспечению
- .NET 6.0 Runtime;
- PostgreSQL 15 или выше;
- Visual Studio 2022 (для разработки).

10. Требования к кодированию
- Использование языка программирования C#;
- Соблюдение стандартов именования (PascalCase, camelCase);
- Наличие комментариев к сложным участкам кода;
- Использование осмысленных имен переменных и методов.

Разработчик: _____ / [Ваше ФИО] /

Руководитель практики: _____ / [ФИО руководителя] /

Дата: 16.03.2026

Приложение 2

Скрипт базы данных

```
mkdir C:\psql_tablespace
psql -U postgres
create tablespace app_space location ‘C:\psql_tablespace’;
create role app with login password ‘123456789’;
create database tebenkova tablespace app_space;
\c tebenkova
create schema app authorization app;
grant all on schema app to app;
grant all on all tables in schema app to app;
grant all privileges on database tebenkova to app;
create table app."PartnerTypes"("id" serial primary key, "name" varchar(100) not null);
create table app."Suppliers"("id" serial primary key, "name" varchar(100) not null, "ContactPerson" varchar(200), "phone" varchar(20), "email" varchar(100), "INN" varchar(20), "address" text, "rating" integer default 0);
create table app."ProductTypes"("id" serial primary key, "name" varchar(100) not null);
create table app."Products"("id" serial primary key, "article" varchar(50) not null, "name" varchar(200) not null, "productTypeId" integer references app."ProductTypes"("id"), "description" text, "unit" varchar(20) not null, "quantityInStock" decimal(18,2) default 0, "MinStockQuantity" decimal(18,2) default 0, "CostPrice" decimal(18,2), "SupplierId" integer references app."Suppliers"("id"), "location" varchar(100));
create table app."Partners"("id" serial primary key, "TypeId" integer not null references app."PartnerTypes"("id"), "name" varchar(200) not null, "legalAddress" text, "INN" varchar(20), "DirectorName" varchar(200), "phone" varchar(20), "email" varchar(100), "LogoPath" text, "Rating" integer not null default 0);
create table app."SalesHistories"("id" serial primary key, "partnerId" integer not null references app."Partners"("id"), "ProductId" integer not null references app."Products"("id"), "SalesDate" date not null, "quantity" integer not null, "salesPrice" decimal(18,2) not null);
create table app."StockReceipts"("id" serial primary key, "ReceiptDate" timestamp not null default now(), "ProductId" integer not null references app."Products"("id"), "quantity" decimal(18,2) not null, "price" decimal(18,2) not null, "SupplierId" integer references app."Suppliers"("id"), "DocumentNumber" varchar(50));
create table app."StockIssues"("id" serial primary key, "IssueDate" timestamp not null default now(), "ProductId" integer not null references app."Products"("id"), "quantity" decimal(18,2) not null, "Reason" varchar(200), "DocumentNumber" varchar(50));
create index IX_Partners_Name on app."Partners"("name");
create index IX_Partners_INN on app."Partners"("INN");
create index IX_Partners_Phone on app."Partners"("phone");
create index IX_Products_Article on app."Products"("article");
create index IX_Products_Name on app."Products"("name");
create index IX_SalesHistories_SaleDate on app."SalesHistories"("SalesDate");
create index IX_SalesHistories_PartnerId on app."SalesHistories"("partnerId");
create index IX_StockReceipts_ReceiptDate on app."StockReceipts"("ReceiptDate");
create index IX_StockIssueDate on app."StockIssues"("IssueDate");
INSERT INTO app."PartnerTypes" ("name") VALUES ('ООО'), ('ЗАО'), ('ИП'), ('АО'),('ПАО');
INSERT INTO app."ProductTypes" ("name") VALUES ('Срезанные цветы'), ('Комнатные растения'), ('Горшки и кашпо'), ('Грунт и удобрения'), ('Упаковка'), ('Декор');
INSERT INTO app."Suppliers" ("name", "ContactPerson", "phone", "email", "rating") VALUES ('Голландские цветы Б.В.', 'Ян Ван Дейк', '+7 (495) 123-45-67', 'info@hollandflowers.ru', 5), ('Тепличный комплекс "Цветущий край"', 'Петрова Анна Николаевна', '+7 (342) 234-56-78', 'sales@cvetkray.ru', 4), ('ООО "Горшок и К"', 'Сидоров Иван Петрович', '+7 (912) 345-67-89', 'gorchok@mail.ru', 4), ('ИП "Упаковка-Сервис"', 'Козлова Елена', '+7 (902) 456-78-90', 'elena@packserv.ru', 3);
```

INSERT INTO app."Products" ("article", "name", "productTypeId", "unit", "quantityInStock", "MinStockQuantity", "SupplierId", "location") VALUES ('ROS-001', 'Роза красная (50 см)', 1, 'шт', 150, 50, 1, 'Холодильник №1'), ('ROS-002', 'Роза белая (60 см)', 1, 'шт', 120, 40, 1, 'Холодильник №1'), ('TUL-001', 'Тюльпан желтый', 1, 'шт', 200, 100, 2, 'Холодильник №2'), ('TUL-002', 'Тюльпан красный', 1, 'шт', 180, 80, 2, 'Холодильник №2'), ('POT-001', 'Горшок керамический d=15см', 3, 'шт', 45, 20, 3, 'Стеллаж А-1'), ('POT-002', 'Горшок пластиковый d=12см', 3, 'шт', 70, 30, 3, 'Стеллаж А-1'), ('SOIL-001', 'Грунт универсальный 5л', 4, 'шт', 30, 15, 2, 'Стеллаж Б-2'), ('SOIL-002', 'Керамзит 2л', 4, 'шт', 25, 10, 2, 'Стеллаж Б-2'), ('DEC-001', 'Лента упаковочная красная', 6, 'шт', 100, 30, 4, 'Стеллаж В-3'), ('DEC-002', 'Бумага упаковочная крафт', 5, 'шт', 50, 20, 4, 'Стеллаж В-3');

INSERT INTO app."Partners" ("TypeId", "name", "legalAddress", "phone", "email", "Rating") VALUES (1, 'ООО "Цветочный рай"', 'г. Пермь, ул. Ленина, 10', '+7 (342) 111-22-33', 'ray@mail.ru', 5), (3, 'ИП Петрова М.И.', 'г. Пермь, ул. Сибирская, 45', '+7 (912) 222-33-44', 'maria@yandex.ru', 4), (1, 'ООО "Флора-Маркет"', 'г. Пермь, ул. Мира, 78', '+7 (342) 333-44-55', 'info@flora.ru', 5), (2, 'ЗАО "Садовод"', 'г. Краснокамск, ул. Садовая, 5', '+7 (342) 444-55-66', 'sadovod@bk.ru', 3), (3, 'ИП Сидоров А.А.', 'г. Пермь, ул. Пушкина, 15', '+7 (912) 555-66-77', 'sidorov@mail.ru', 2);

INSERT INTO app."SalesHistories" ("partnerId", "ProductId", "quantity", "SalesDate", "salesPrice") VALUES (1, 1, 100, '2026-02-10', 80.00), (1, 2, 50, '2026-02-15', 90.00), (1, 5, 20, '2026-02-20', 250.00), (2, 3, 200, '2026-02-12', 45.00), (2, 4, 150, '2026-02-18', 45.00), (2, 9, 30, '2026-02-25', 35.00), (3, 1, 300, '2026-03-01', 75.00), (3, 2, 250, '2026-03-01', 85.00), (3, 6, 50, '2026-03-05', 60.00), (4, 7, 40, '2026-02-28', 120.00), (5, 8, 15, '2026-03-02', 45.00);

INSERT INTO app."StockReceipts" ("ReceiptDate", "ProductId", "quantity", "price", "SupplierId", "DocumentNumber") VALUES ('2026-03-01 10:30:00', 1, 200, 60.00, 1, 'INV-001'), ('2026-03-01 10:30:00', 2, 150, 70.00, 1, 'INV-001'), ('2026-03-02 14:20:00', 3, 300, 30.00, 2, 'INV-002'), ('2026-03-02 14:20:00', 4, 250, 30.00, 2, 'INV-002'), ('2026-03-03 09:15:00', 5, 60, 180.00, 3, 'INV-003'), ('2026-03-03 09:15:00', 6, 80, 40.00, 3, 'INV-003');

INSERT INTO app."StockIssues" ("IssueDate", "ProductId", "quantity", "Reason", "DocumentNumber") VALUES ('2026-03-02 15:30:00', 1, 50, 'Продажа ООО "Цветочный рай"', 'SALE-001'), ('2026-03-02 15:30:00', 2, 30, 'Продажа ООО "Цветочный рай"', 'SALE-001'), ('2026-03-03 11:20:00', 3, 100, 'Продажа ИП Петрова М.И.', 'SALE-002'), ('2026-03-04 10:00:00', 5, 10, 'Списание (брак)', 'WRITE-OFF-001');

```
CREATE OR REPLACE FUNCTION app.CalculateDiscount(partner_id INTEGER)
RETURNS INTEGER AS $$
DECLARE
  total_sales DECIMAL(18,2);
  discount INTEGER;
BEGIN
  -- Считаем общую сумму продаж партнера
  SELECT COALESCE(SUM(sh.Quantity * sh.SalePrice), 0) INTO total_sales
  FROM app."SalesHistories" sh
  WHERE sh."PartnerId" = partner_id;
  -- Определяем скидку по правилам
  IF total_sales >= 300000 THEN
    discount := 15;
  ELSIF total_sales >= 50000 THEN
    discount := 10;
  ELSIF total_sales >= 10000 THEN
    discount := 5;
  ELSE
    discount := 0;
  END IF;
  RETURN discount;
END;
$$ LANGUAGE plpgsql;
```

Код программы

Код класса AppDbContext.cs

```
namespace tebenkova_Data
{
  public class AppDbContext : DbContext
  {
    // DbSet для всех таблиц
    public DbSet<PartnerType> PartnerTypes { get; set; }
    public DbSet<Partner> Partners { get; set; }
    public DbSet<ProductType> ProductTypes { get; set; }
    public DbSet<Product> Products { get; set; }
    public DbSet<Supplier> Suppliers { get; set; }
    public DbSet<SaleHistory> SalesHistories { get; set; }
    public DbSet<StockReceipt> StockReceipts { get; set; }
    public DbSet<StockIssue> StockIssues { get; set; }
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
optionsBuilder.UseNpgsql("Host=localhost;Port=5432;Database=tebenkova;Username=app;Password=123456789");
    }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
      base.OnModelCreating(modelBuilder);
      // Устанавливаем схему по умолчанию
      modelBuilder.HasDefaultSchema("app");
      // Настройка связей для Partner
      modelBuilder.Entity<Partner>()
        .HasOne(p => p.PartnerType)
        .WithMany(pt => pt.Partners)
        .HasForeignKey(p => p.TypeId)
        .OnDelete(DeleteBehavior.Restrict);
      // Настройка связей для Product
      modelBuilder.Entity<Product>()
        .HasOne(p => p.ProductType)
        .WithMany(pt => pt.Products)
        .HasForeignKey(p => p.ProductTypeId)
        .OnDelete(DeleteBehavior.Restrict);
      modelBuilder.Entity<Product>()
        .HasOne(p => p.Supplier)
        .WithMany(s => s.Products)
        .HasForeignKey(p => p.SupplierId)
        .OnDelete(DeleteBehavior.Restrict);
      // Настройка связей для SaleHistory
      modelBuilder.Entity<SaleHistory>()
        .HasOne(sh => sh.Partner)
        .WithMany(p => p.SalesHistories)
        .HasForeignKey(sh => sh.PartnerId)
        .OnDelete(DeleteBehavior.Cascade);
      modelBuilder.Entity<SaleHistory>()
        .HasOne(sh => sh.Product)
        .WithMany(p => p.SalesHistories)
        .HasForeignKey(sh => sh.ProductId)
        .OnDelete(DeleteBehavior.Restrict);
      // Настройка связей для StockReceipt
      modelBuilder.Entity<StockReceipt>()
        .HasOne(sr => sr.Product)
        .WithMany(p => p.StockReceipts)
        .HasForeignKey(sr => sr.ProductId)
        .OnDelete(DeleteBehavior.Restrict);
      modelBuilder.Entity<StockReceipt>()
        .HasOne(sr => sr.Supplier)
        .WithMany(s => s.StockReceipts)
        .HasForeignKey(sr => sr.SupplierId)
        .OnDelete(DeleteBehavior.Restrict);
      // Настройка связей для StockIssue
      modelBuilder.Entity<StockIssue>()
        .HasOne(si => si.Product)
        .WithMany(p => p.StockIssues)
        .HasForeignKey(si => si.ProductId)
        .OnDelete(DeleteBehavior.Restrict);}}}
```

Код класса PartnerService.cs

```
namespace tebenkova_Services.Services
{
  public class PartnerService : IPartnerService
  {
    private readonly IPartnerRepository _partnerRepository;
    private readonly ISaleHistoryRepository _saleHistoryRepository;
    public PartnerService(
      IPartnerRepository partnerRepository,
      ISaleHistoryRepository saleHistoryRepository)
    {
      _partnerRepository = partnerRepository;
      _saleHistoryRepository = saleHistoryRepository;
    }
    public async Task<IEnumerable<Partner>> GetAllPartnersAsync()
    {
      return await _partnerRepository.GetAllAsync();
    }
    public async Task<Partner> GetPartnerByIdAsync(int id)
    {
      return await _partnerRepository.GetByIdAsync(id);
    }
    public async Task<Partner> GetPartnerDetailAsync(int id)
    {
      return await _partnerRepository.GetByIdWithDetailsAsync(id);
    }
    public async Task<Partner> CreatePartnerAsync(Partner partner)
    {
      if (string.IsNullOrWhiteSpace(partner.Name))
        throw new ArgumentException("Наименование партнера обязательно");
      if (partner.Rating < 0 || partner.Rating > 10)
        throw new ArgumentException("Рейтинг должен быть от 0 до 10");
      if (!string.IsNullOrWhiteSpace(partner.INN) && !ValidationHelper.IsValidINN(partner.INN))
        throw new ArgumentException("Неверный формат ИНН");
      if (!string.IsNullOrWhiteSpace(partner.Phone) && !ValidationHelper.IsValidPhone(partner.Phone))
        throw new ArgumentException("Неверный формат телефона");
      if (!string.IsNullOrWhiteSpace(partner.Email) && !ValidationHelper.IsValidEmail(partner.Email))
        throw new ArgumentException("Неверный формат email");
      return await _partnerRepository.AddAsync(partner);
    }
    public async Task UpdatePartnerAsync(Partner partner)
    {
      if (!await _partnerRepository.ExistsAsync(partner.Id))
        throw new KeyNotFoundException($"Партнер с ID {partner.Id} не найден");
      await _partnerRepository.UpdateAsync(partner);
    }
    public async Task DeletePartnerAsync(int id)
    {
      await _partnerRepository.DeleteAsync(id);
    }
    public async Task<int> CalculateDiscountAsync(int partnerId)
    {
      var sales = await _saleHistoryRepository.GetByPartnerIdAsync(partnerId);
      return DiscountHelper.CalculateDiscount(sales);
    }
    public async Task<IEnumerable<Partner>> SearchPartnersAsync(string searchTerm)
    {
      return await _partnerRepository.SearchAsync(searchTerm);
    }
    public async Task<PartnerStatisticsDto> GetPartnerStatisticsAsync(int partnerId)
    {
      var partner = await _partnerRepository.GetByIdWithDetailsAsync(partnerId);
      var sales = await _saleHistoryRepository.GetByPartnerIdAsync(partnerId);
      return new PartnerStatisticsDto
      {
        PartnerId = partner.Id,
        PartnerName = partner.Name,
        TotalSales = sales.Sum(s => s.Quantity * s.SalePrice),
        TotalTransactions = sales.Count(),
        AverageCheck = sales.Any() ? sales.Average(s => s.Quantity * s.SalePrice) : 0,
        FirstPurchaseDate = sales.Any() ? sales.Min(s => s.SaleDate) : (DateTime?)null,
        LastPurchaseDate = sales.Any() ? sales.Max(s => s.SaleDate) : (DateTime?)null,
        Discount = DiscountHelper.CalculateDiscount(sales)
      };
    }}}
```

Код класса ProductService.cs

```
namespace tebenkova_Services.Services
{
    public class ProductService
    {
        private readonly IProductRepository _productRepository;
        private readonly ISupplierRepository _supplierRepository;
        public ProductService(
            IProductRepository productRepository,
            ISupplierRepository supplierRepository)
        {
            _productRepository = productRepository;
            _supplierRepository = supplierRepository;
        }
        public async Task<IEnumerable<Product>> GetAllProductsAsync()
        {
            return await _productRepository.GetAllAsync();
        }
        public async Task<Product> GetProductByIdAsync(int id)
        {
            return await _productRepository.GetByIdWithDetailsAsync(id);
        }
        public async Task<Product> CreateProductAsync(Product product)
        {
            if (string.IsNullOrWhiteSpace(product.Article))
                throw new ArgumentException("Артикул обязателен");
            if (string.IsNullOrWhiteSpace(product.Name))
                throw new ArgumentException("Наименование товара обязательно");
            if (product.QuantityInStock < 0)
                throw new ArgumentException("Количество на складе не может быть отрицательным");
            return await _productRepository.AddAsync(product);
        }
        public async Task UpdateProductAsync(Product product)
        {
            await _productRepository.UpdateAsync(product);
        }
        public async Task DeleteProductAsync(int id)
        {
            await _productRepository.DeleteAsync(id);
        }
        public async Task<IEnumerable<Product>> GetLowStockProductsAsync()
        {
            return await _productRepository.GetLowStockAsync();
        }
        public async Task<StockSummaryDto> GetStockSummaryAsync()
        {
            var products = await _productRepository.GetAllAsync();
            return new StockSummaryDto
            {
                TotalProducts = products.Count(),
                TotalItemsInStock = products.Sum(p => p.QuantityInStock),
                LowStockItems = products.Count(p => p.QuantityInStock <= p.MinStockQuantity),
                OutOfStockItems = products.Count(p => p.QuantityInStock <= 0),
                TotalStockValue = products.Sum(p => p.QuantityInStock * (p.CostPrice ?? 0)),
                ProductsByType = products
                    .GroupBy(p => p.ProductType?.Name ?? "Без типа")
                    .ToDictionary(g => g.Key, g => g.Count())
            };
        }
    }
}
```

Код класса SupplierService.cs

```
namespace tebenkova_Services.Services
{
    public class SupplierService
    {
        private readonly ISupplierRepository _supplierRepository;
        private readonly IProductRepository _productRepository;

        public SupplierService(
            ISupplierRepository supplierRepository,
            IProductRepository productRepository)
        {
```

```
        _supplierRepository = supplierRepository;
        _productRepository = productRepository;
    }

    public async Task<IEnumerable<Supplier>> GetAllSuppliersAsync()
    {
        return await _supplierRepository.GetAllAsync();
    }

    public async Task<Supplier> GetSupplierByIdAsync(int id)
    {
        return await _supplierRepository.GetByIdWithProductsAsync(id);
    }

    public async Task<Supplier> CreateSupplierAsync(Supplier supplier)
    {
        if (string.IsNullOrWhiteSpace(supplier.Name))
            throw new ArgumentException("Наименование поставщика обязательно");

        if (!string.IsNullOrWhiteSpace(supplier.INN) && !ValidationHelper.IsValidINN(supplier.INN))
            throw new ArgumentException("Неверный формат ИНН");

        return await _supplierRepository.AddAsync(supplier);
    }

    public async Task UpdateSupplierAsync(Supplier supplier)
    {
        await _supplierRepository.UpdateAsync(supplier);
    }

    public async Task DeleteSupplierAsync(int id)
    {
        var products = await _productRepository.GetBySupplierIdAsync(id);
        if (products.Any())
            throw new InvalidOperationException("Нельзя удалить поставщика, у которого есть товары");

        await _supplierRepository.DeleteAsync(id);
    }

    public async Task<SupplierStatisticsDto> GetSupplierStatisticsAsync(int supplierId)
    {
        var supplier = await _supplierRepository.GetByIdWithProductsAsync(supplierId);
        var products = supplier.Products ?? new List<Product>();

        return new SupplierStatisticsDto
        {
            SupplierId = supplier.Id,
            SupplierName = supplier.Name,
            TotalProducts = products.Count(),
            TotalStockValue = products.Sum(p => p.QuantityInStock * (p.CostPrice ?? 0)),
            AverageProductPrice = products.Any() ? products.Average(p => p.CostPrice ?? 0) : 0,
            ProductsByType = products
                .GroupBy(p => p.ProductType?.Name ?? "Без типа")
                .ToDictionary(g => g.Key, g => g.Count()),
            Rating = supplier.Rating
        };
    }
  }
}
```

Приложение 4

Демонстрация работы приложения

1. Добавление нового партнера

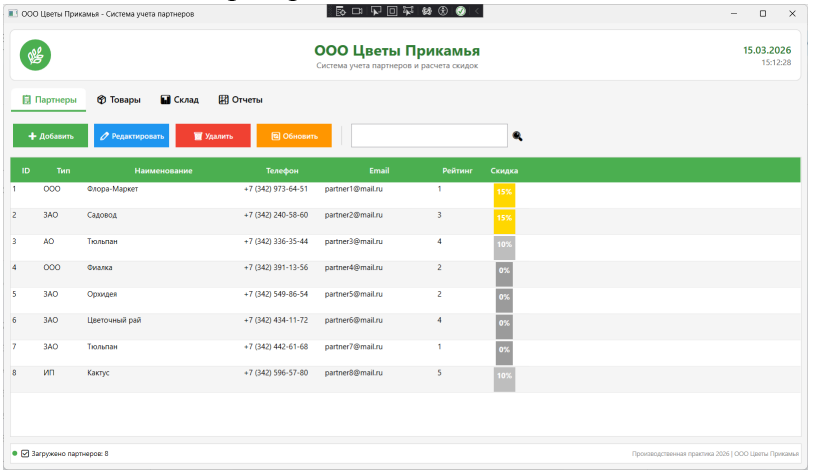

Рисунок 12 - Вкладка “Партнеры”

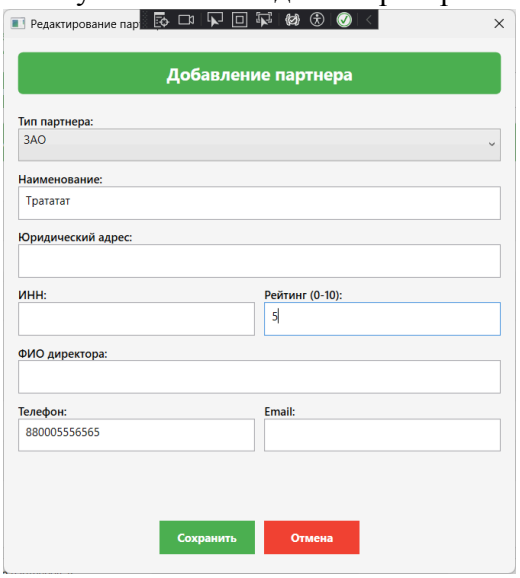

Рисунок 13 - Добавление нового партнера

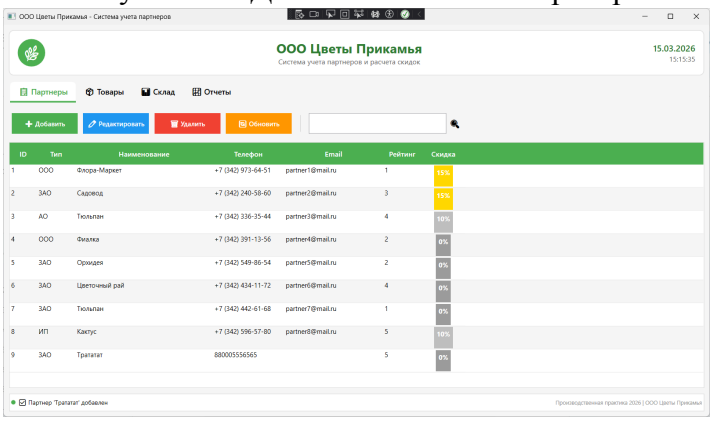

Рисунок 14 - Результат добавления партнера

2. Редактирование существующего партнера

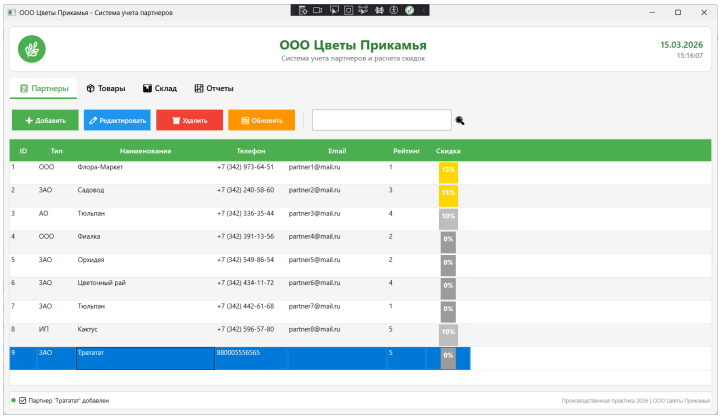

Рисунок 15 - Выбор партнера для редактирования

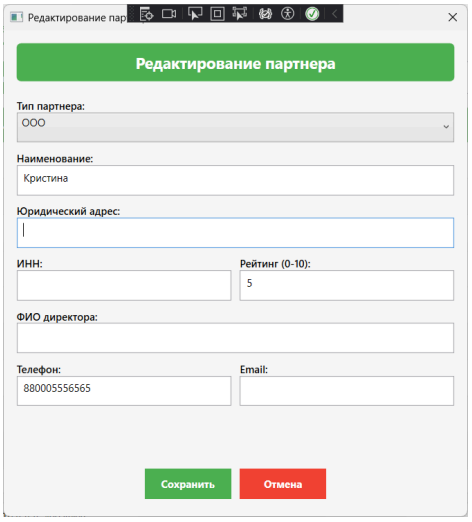

Рисунок 16 - Редактирование партнера

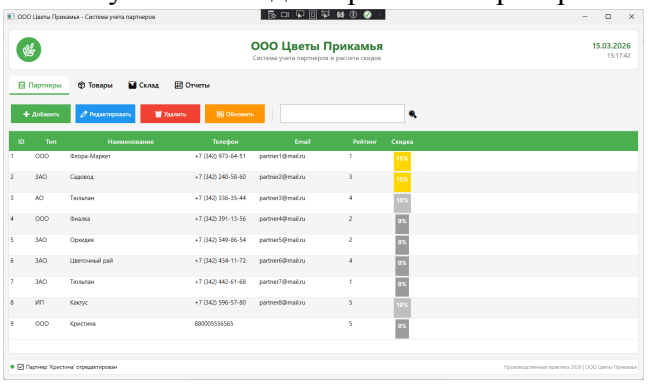

Рисунок 17 - Результат редактирования партнера

3. Удаление партнера

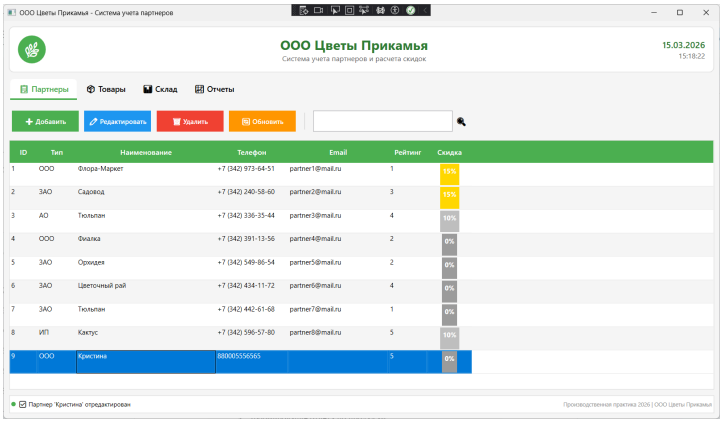

Рисунок 18 - Выбор партнера для удаления

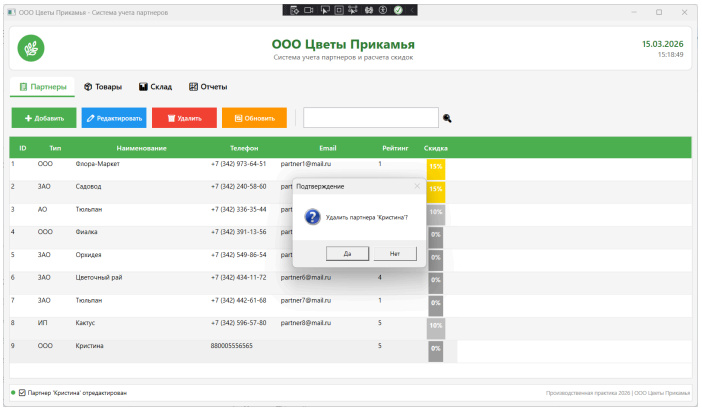

Рисунок 19 - Подтверждение удаления

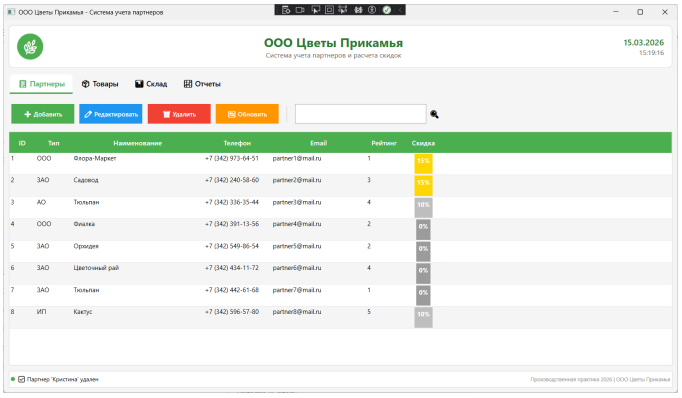

Рисунок 20 - Результат удаления партнера

4. Добавление нового товара

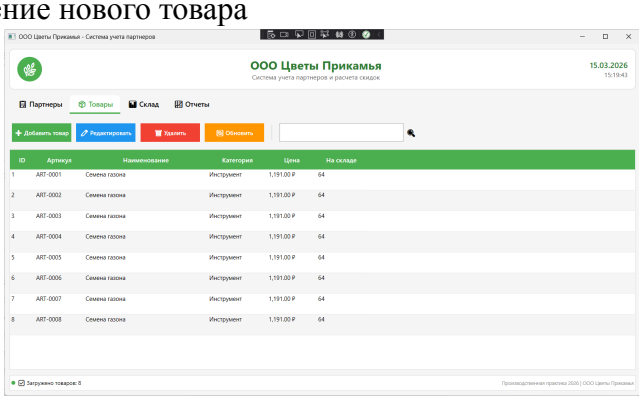

Рисунок 21 - Вкладка “Товары”

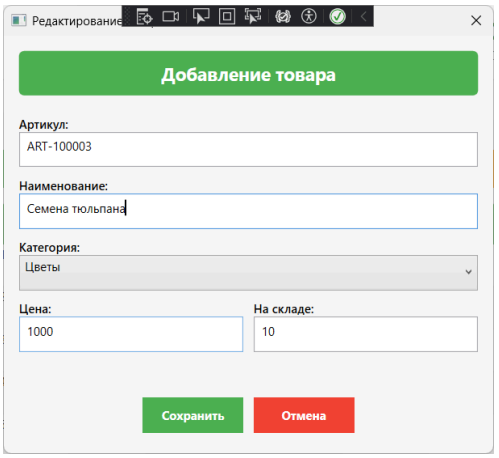

Рисунок 22 - Добавление товара

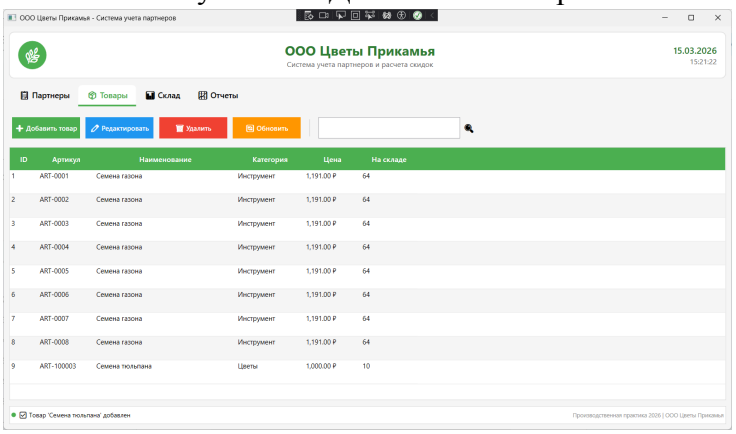

Рисунок 23 - Результат добавления товара

5. Редактирование существующего товара

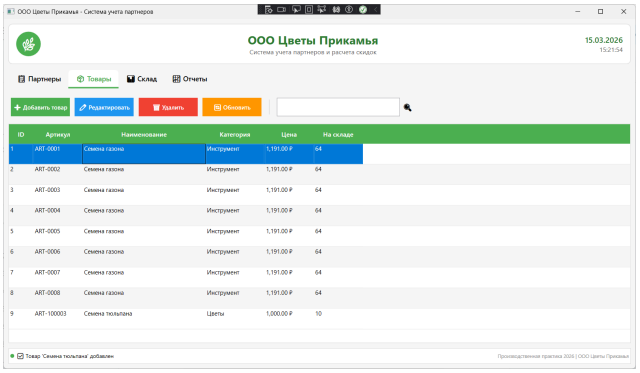

Рисунок 24 - Выбор товара для редактирования

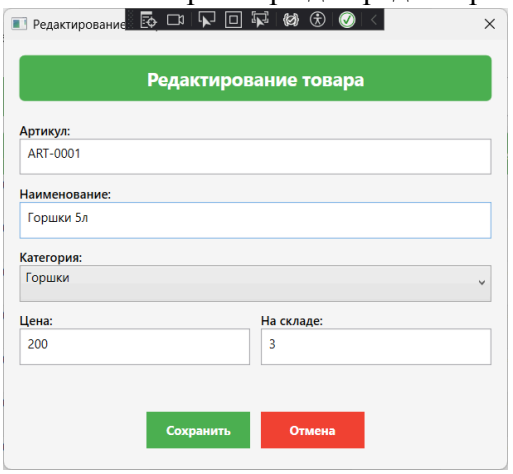

Рисунок 25 - Редактирование товара

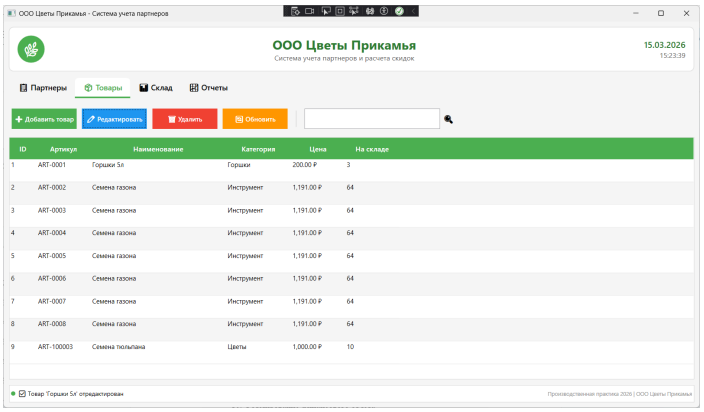

Рисунок 26 - Результат редактирования товара

6. Удаление товара

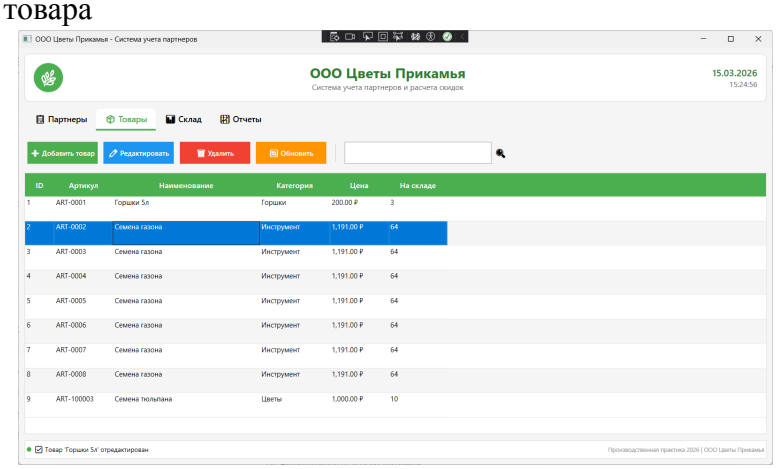

Рисунок 27 - Выбор товара для удаления

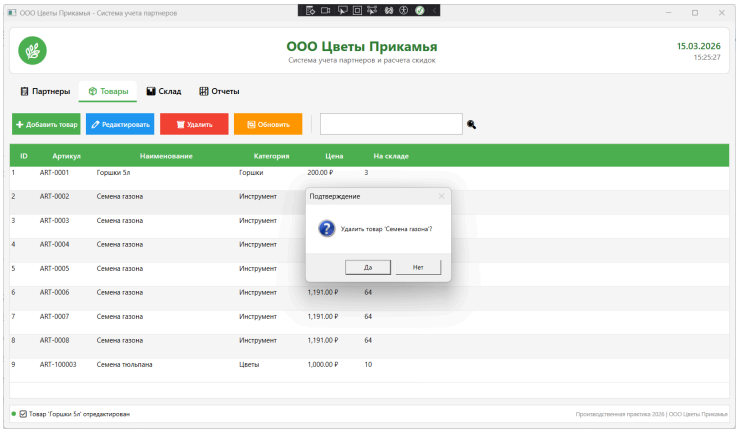

Рисунок 28 - Подтверждение удаления

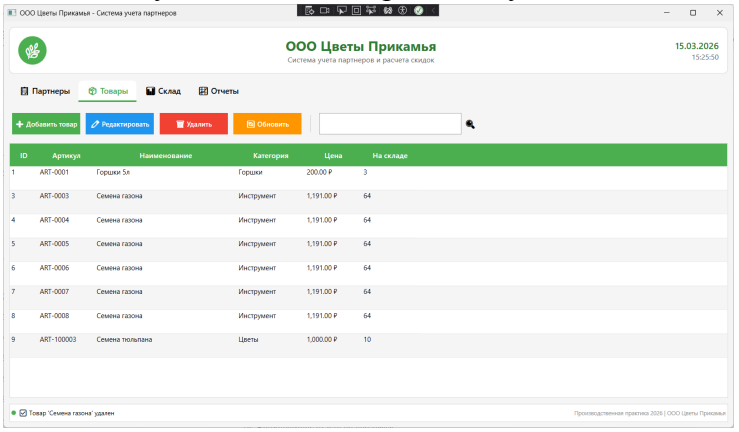

Рисунок 29 - Результат удаления

7. Поиск партнера

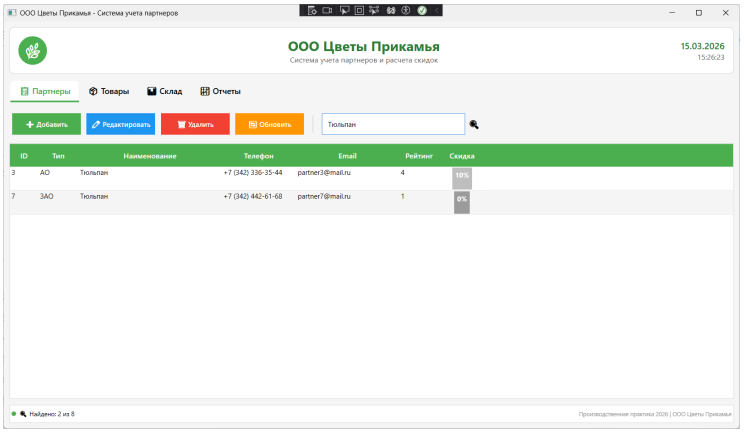

Рисунок 30 - Поиск партнера

8. Поиск товара

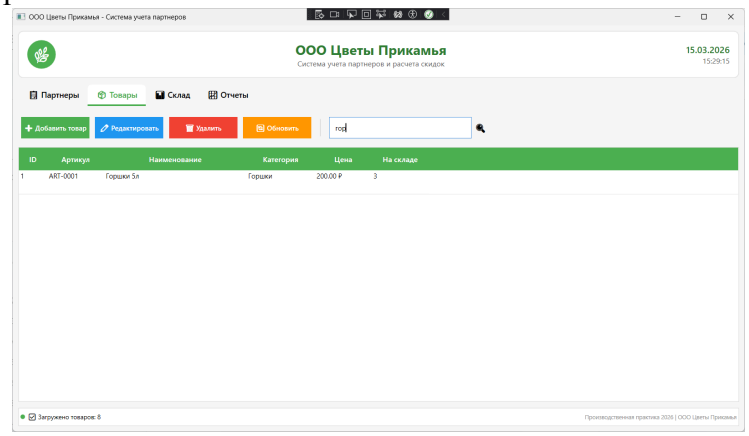

Рисунок 31 - Поиск товара

9. Вкладка “Склад”

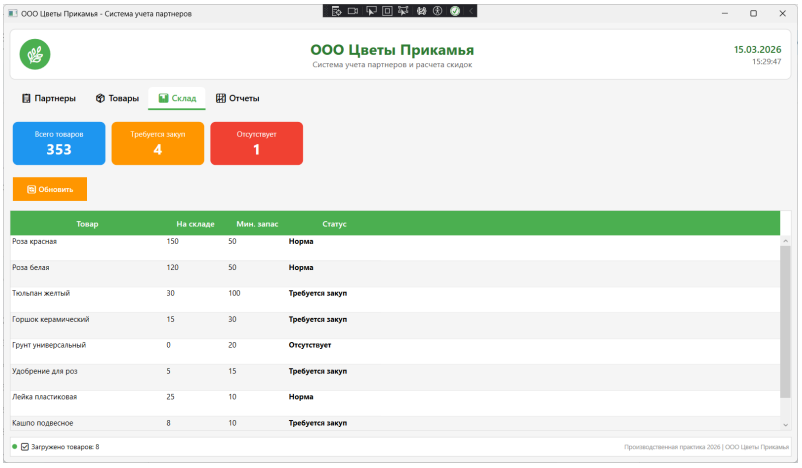

Рисунок 32 - Вкладка “Склад”

10. Формирование отчета по продажам

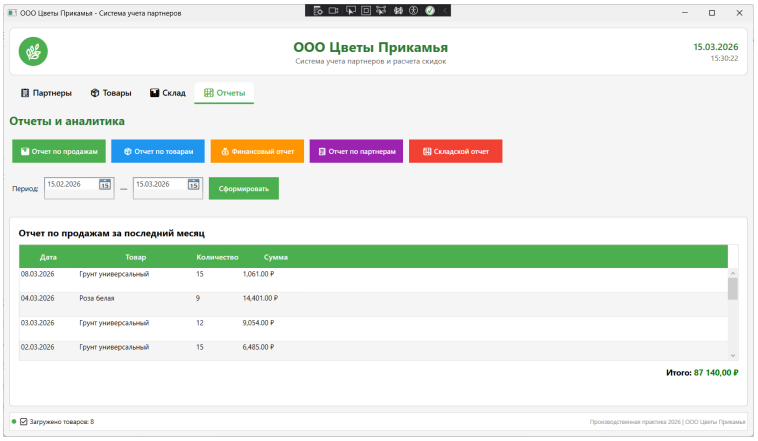

Рисунок 33 - Отчет по продажам

11. Формирование отчета по товарам

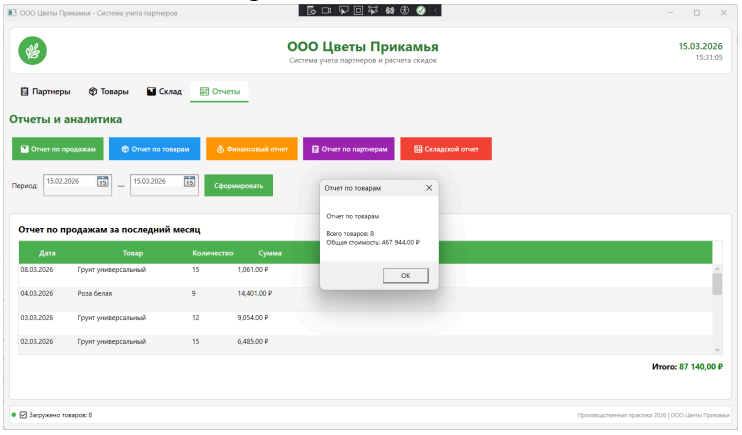

Рисунок 34 - Отчет по товарам

12. Формирование финансового отчета

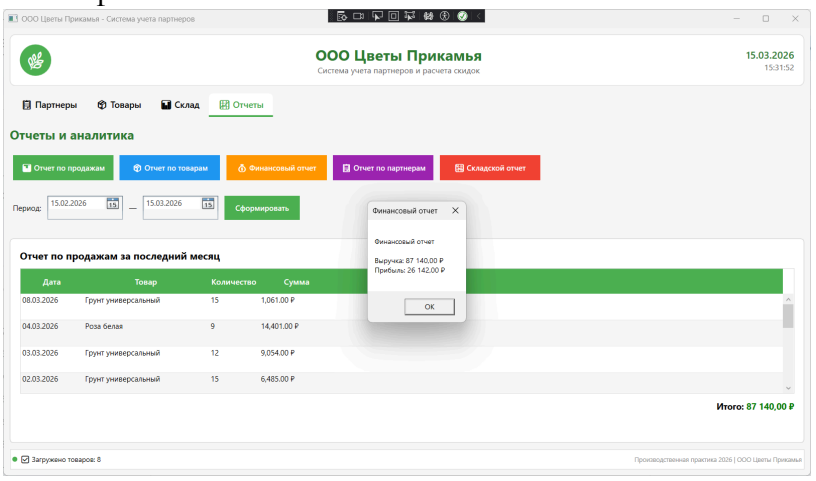

Рисунок 35 - Финансовый отчет

13. Формирование отчета по партнерам

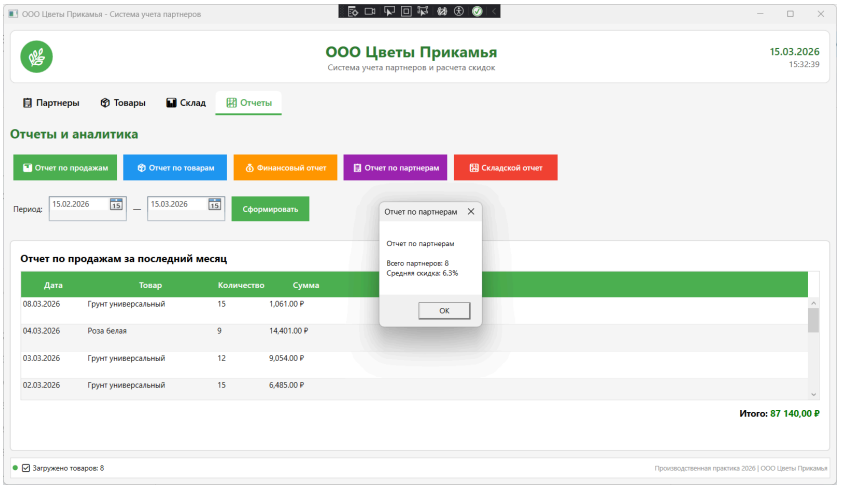

Рисунок 36 - Отчет по партнерам

14. Формирование складского отчета

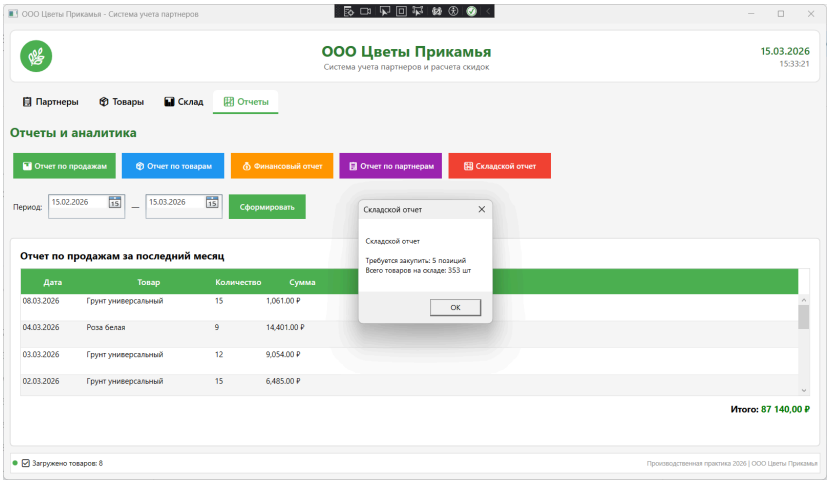

Рисунок 37 - Складской отчет

Приложение 5

Unit-тестирование

Код класса PartnerServiceTest.cs

```
namespace tebenkova_Tests.Services
{
  [TestClass]
  public class PartnerServiceTests
  {
    private Mock<IPartnerRepository> _mockPartnerRepository;
    private Mock<ISaleHistoryRepository> _mockSaleHistoryRepository;
    private PartnerService _service;

    [TestInitialize]
    public void Setup()
    {
      _mockPartnerRepository = new Mock<IPartnerRepository>();
      _mockSaleHistoryRepository = new Mock<ISaleHistoryRepository>();
      _service = new PartnerService(_mockPartnerRepository.Object, _mockSaleHistoryRepository.Object);
    }
    [TestMethod]
    public async Task GetAllPartnersAsync_ShouldReturnAllPartners()
    {
      var partners = new List<Partner>
      {
        new Partner { Id = 1, Name = "Партнер 1" },
        new Partner { Id = 2, Name = "Партнер 2" }
      };
      _mockPartnerRepository.Setup(r => r.GetAllAsync()).ReturnsAsync(partners);

      var result = await _service.GetAllPartnersAsync();

      Assert.AreEqual(2, result.Count());
      Assert.AreEqual("Партнер 1", result.First().Name);
    }
    [TestMethod]
    public async Task GetPartnerByIdAsync_WithValidId_ShouldReturnPartner()
    {
      var partner = new Partner { Id = 1, Name = "Партнер 1" };
      _mockPartnerRepository.Setup(r => r.GetByIdAsync(1)).ReturnsAsync(partner);

      var result = await _service.GetPartnerByIdAsync(1);

      Assert.IsNotNull(result);
      Assert.AreEqual(1, result.Id);
      Assert.AreEqual("Партнер 1", result.Name);
    }
    [TestMethod]
    public async Task GetPartnerByIdAsync_WithInvalidId_ShouldReturnNull()
    {
      _mockPartnerRepository.Setup(r => r.GetByIdAsync(999)).ReturnsAsync((Partner)null);

      var result = await _service.GetPartnerByIdAsync(999);

      Assert.IsNull(result);
    }
    [TestMethod]
    public async Task GetPartnerDetailAsync_ShouldReturnPartnerWithDetails()
    {
      var partner = new Partner
      {
        Id = 1,
        Name = "Партнер 1",
        SalesHistories = new List<SaleHistory>
        {
          new SaleHistory { Id = 1, Quantity = 10, SalePrice = 100 }
        }
      };
      _mockPartnerRepository.Setup(r => r.GetByIdWithDetailsAsync(1)).ReturnsAsync(partner);

      var result = await _service.GetPartnerDetailAsync(1);

      Assert.IsNotNull(result);
      Assert.AreEqual(1, result.SalesHistories.Count);
    }
```

```
[TestMethod]
public async Task CreatePartnerAsync_WithValidData_ShouldCreatePartner()
{
    var partner = new Partner
    {
        Name = "Новый партнер",
        Rating = 5,
        INN = "1234567890",
        Phone = "+7 (342) 123-45-67",
        Email = "test@test.com"
    };
    var createdPartner = new Partner
    {
        Id = 1,
        Name = "Новый партнер",
        Rating = 5
    };
    _mockPartnerRepository.Setup(r => r.AddAsync(It.IsAny<Partner>())).ReturnsAsync(createdPartner);

    var result = await _service.CreatePartnerAsync(partner);

    Assert.IsNotNull(result);
    Assert.AreEqual(1, result.Id);
    Assert.AreEqual("Новый партнер", result.Name);
}
[TestMethod]
public async Task UpdatePartnerAsync_WithValidData_ShouldUpdatePartner()
{
    var partner = new Partner { Id = 1, Name = "Обновленный партнер" };
    _mockPartnerRepository.Setup(r => r.ExistsAsync(1)).ReturnsAsync(true);
    _mockPartnerRepository.Setup(r => r.UpdateAsync(It.IsAny<Partner>())).Returns(Task.CompletedTask);

    await _service.UpdatePartnerAsync(partner);

    _mockPartnerRepository.Verify(r => r.UpdateAsync(partner), Times.Once);
}
[TestMethod]
public async Task DeletePartnerAsync_ShouldDeletePartner()
{
    _mockPartnerRepository.Setup(r => r.DeleteAsync(1)).Returns(Task.CompletedTask);

    await _service.DeletePartnerAsync(1);

    _mockPartnerRepository.Verify(r => r.DeleteAsync(1), Times.Once);
}
[TestMethod]
public async Task CalculateDiscountAsync_WithNoSales_ShouldReturnZero()
{
    _mockSaleHistoryRepository.Setup(r => r.GetByPartnerIdAsync(1))
        .ReturnsAsync(new List<SaleHistory>());

    var result = await _service.CalculateDiscountAsync(1);

    Assert.AreEqual(0, result);
}
[TestMethod]
public async Task CalculateDiscountAsync_WithSalesLess10000_ShouldReturnZero()
{
    var sales = new List<SaleHistory>
    {
        new SaleHistory { Quantity = 1, SalePrice = 5000 }
    };
    _mockSaleHistoryRepository.Setup(r => r.GetByPartnerIdAsync(1)).ReturnsAsync(sales);

    var result = await _service.CalculateDiscountAsync(1);

    Assert.AreEqual(0, result);
}
[TestMethod]
public async Task CalculateDiscountAsync_WithSalesBetween10000And50000_ShouldReturn5()
{
    var sales = new List<SaleHistory>
    {
        new SaleHistory { Quantity = 2, SalePrice = 10000 }
    };
```

```
            _mockSaleHistoryRepository.Setup(r => r.GetByPartnerIdAsync(1)).ReturnsAsync(sales);

            var result = await _service.CalculateDiscountAsync(1);

            Assert.AreEqual(5, result);
        }
        [TestMethod]
        public async Task CalculateDiscountAsync_WithSalesBetween50000And300000_ShouldReturn10()
        {
            var sales = new List<SaleHistory>
            {
                new SaleHistory { Quantity = 5, SalePrice = 20000 }
            };
            _mockSaleHistoryRepository.Setup(r => r.GetByPartnerIdAsync(1)).ReturnsAsync(sales);

            var result = await _service.CalculateDiscountAsync(1);

            Assert.AreEqual(10, result);
        }
        [TestMethod]
        public async Task CalculateDiscountAsync_WithSalesMore300000_ShouldReturn15()
        {
            var sales = new List<SaleHistory>
            {
                new SaleHistory { Quantity = 10, SalePrice = 40000 }
            };
            _mockSaleHistoryRepository.Setup(r => r.GetByPartnerIdAsync(1)).ReturnsAsync(sales);

            var result = await _service.CalculateDiscountAsync(1);

            Assert.AreEqual(15, result);
        }
        [TestMethod]
        public async Task SearchPartnersAsync_WithSearchTerm_ShouldReturnMatchingPartners()
        {
            var allPartners = new List<Partner>
            {
                new Partner { Id = 1, Name = "ООО Ромашка" },
                new Partner { Id = 2, Name = "ИП Петров" }
            };
            var filteredPartners = allPartners.Where(p => p.Name.Contains("Ромашка")).ToList();

            _mockPartnerRepository.Setup(r => r.SearchAsync("Ромашка"))
                .ReturnsAsync(filteredPartners);

            var result = await _service.SearchPartnersAsync("Ромашка");

            Assert.AreEqual(1, result.Count());
            Assert.AreEqual("ООО Ромашка", result.First().Name);
        }
    }
}
```

Код класса ProductServiceTest.cs

```
namespace tebenkova_Tests.Services
{
    [TestClass]
    public class ProductServiceTests
    {
        private Mock<IProductRepository> _mockProductRepository;
        private Mock<ISupplierRepository> _mockSupplierRepository;
        private ProductService _service;
        [TestInitialize]
        public void Setup()
        {
            _mockProductRepository = new Mock<IProductRepository>();
            _mockSupplierRepository = new Mock<ISupplierRepository>();
            _service = new ProductService(_mockProductRepository.Object, _mockSupplierRepository.Object);
        }
        [TestMethod]
        public async Task GetAllProductsAsync_ShouldReturnAllProducts()
        {
            var products = new List<Product>
            {
                new Product { Id = 1, Name = "Розы", Article = "ROS001" },
                new Product { Id = 2, Name = "Тюльпаны", Article = "TUL001" }
```

```
            };
            _mockProductRepository.Setup(r => r.GetAllAsync()).ReturnsAsync(products);
            var result = await _service.GetAllProductsAsync();
            Assert.AreEqual(2, result.Count());
            Assert.AreEqual("Розы", result.First().Name);
        }
        [TestMethod]
        public async Task GetProductByIdAsync_WithValidId_ShouldReturnProduct()
        {
            var product = new Product
            {
                Id = 1,
                Name = "Розы",
                Article = "ROS001",
                ProductType = new ProductType { Name = "Цветы" },
                Supplier = new Supplier { Name = "Поставщик 1" }
            };
            _mockProductRepository.Setup(r => r.GetByIdWithDetailsAsync(1)).ReturnsAsync(product);
            var result = await _service.GetProductByIdAsync(1);
            Assert.IsNotNull(result);
            Assert.AreEqual(1, result.Id);
            Assert.AreEqual("Розы", result.Name);
        }
        [TestMethod]
        public async Task CreateProductAsync_WithValidData_ShouldCreateProduct()
        {
            var product = new Product
            {
                Article = "NEW001",
                Name = "Новый товар",
                QuantityInStock = 100
            };
            var createdProduct = new Product
            {
                Id = 1,
                Article = "NEW001",
                Name = "Новый товар",
                QuantityInStock = 100
            };
            _mockProductRepository.Setup(r => r.AddAsync(It.IsAny<Product>())).ReturnsAsync(createdProduct);
            var result = await _service.CreateProductAsync(product);
            Assert.IsNotNull(result);
            Assert.AreEqual(1, result.Id);
            Assert.AreEqual("NEW001", result.Article);
        }
        [TestMethod]
        public async Task GetLowStockProductsAsync_ShouldReturnProductsBelowMinStock()
        {
            var products = new List<Product>
            {
                new Product { Id = 1, Name = "Товар 1", QuantityInStock = 5, MinStockQuantity = 10 },
                new Product { Id = 2, Name = "Товар 2", QuantityInStock = 20, MinStockQuantity = 10 },
                new Product { Id = 3, Name = "Товар 3", QuantityInStock = 0, MinStockQuantity = 5 }
            };
                _mockProductRepository.Setup(r => r.GetLowStockAsync()).ReturnsAsync(products.Where(p =>
p.QuantityInStock <= p.MinStockQuantity));
            var result = await _service.GetLowStockProductsAsync();
            Assert.AreEqual(2, result.Count());
        }
        [TestMethod]
        public async Task GetStockSummaryAsync_ShouldReturnCorrectSummary()
        {
            var products = new List<Product>
            {
                new Product
                {
                    Id = 1,
                    Name = "Товар 1",
                    QuantityInStock = 50,
                    MinStockQuantity = 10,
                    CostPrice = 100,
                    ProductType = new ProductType { Name = "Тип 1" }
                },
                new Product
                {
```

```
                Id = 2,
                Name = "Товар 2",
                QuantityInStock = 5,
                MinStockQuantity = 20,
                CostPrice = 200,
                ProductType = new ProductType { Name = "Тип 1" }
            },
            new Product
            {
                Id = 3,
                Name = "Товар 3",
                QuantityInStock = 0,
                MinStockQuantity = 10,
                CostPrice = 300,
                ProductType = new ProductType { Name = "Тип 2" }
            }
        };
        _mockProductRepository.Setup(r => r.GetAllAsync()).ReturnsAsync(products);
        var result = await _service.GetStockSummaryAsync();
        Assert.AreEqual(3, result.TotalProducts);
        Assert.AreEqual(55, result.TotalItemsInStock);
        Assert.AreEqual(2, result.LowStockItems);
        Assert.AreEqual(1, result.OutOfStockItems);
        Assert.AreEqual(50 * 100 + 5 * 200 + 0 * 300, result.TotalStockValue);
        Assert.AreEqual(2, result.ProductsByType["Тип 1"]);
        Assert.AreEqual(1, result.ProductsByType["Тип 2"]);
    }
  }
}
```

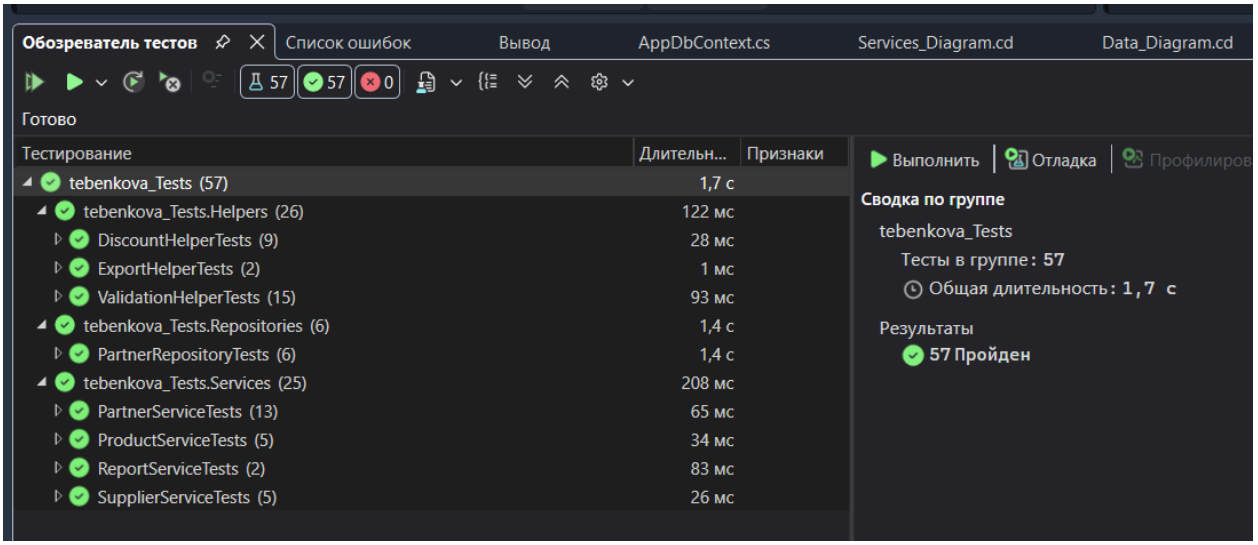

Рисунок 38 - Результат выполнения тестов